

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Теория графов
Отчет о выполнении лабораторных работ
«Алгоритмы на графах»
Вариант 32

Студент,
группы 5130201/40003

_____ Лебедев К. О.

Руководитель,
преподаватель

_____ Востров А. В.

«_____» _____ 20__ г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Математическое описание	7
1.1 Определение графа	7
1.2 Распределение Пирсона	7
1.3 Расстояния	8
1.4 Эксцентриситет и центр	9
1.5 Обходы графов	9
1.6 Выявление маршрутов с заданным количеством рёбер и матри- ца смежности	10
1.7 Метод Шимбелла	10
1.8 Обход в глубину (DFS)	11
1.9 Алгоритм Дейкстры	12
1.10 Поток в сети	12
1.11 Алгоритм Форда-Фалкерсона	13
1.12 Задача нахождения заданного потока минимальной стоимости .	14
1.13 Матрица Кирхгофа	16
1.14 Кратчайший остов	16
1.15 Алгоритм Краскала	16
1.16 Код Прюфера	17
1.17 Нахождение максимального независимого множества вершин . .	18
1.18 Эйлеровы циклы и графы	19
1.19 Фундаментальная система разрезов	20
2 Особенности реализации программы	21
2.1 Структура данных MyGraph	21
2.2 Генератор случайных чисел	21
2.3 Генерация степеней вершин	22
2.4 Построение графа	23
2.5 Поиск эксцентриситета и центра графа	24
2.6 Метод Шимбелла	26
2.7 Подсчет маршрутов	28
2.8 Поиск в глубину (DFS)	29
2.9 Алгоритм Дейкстры	30
2.10 Алгоритм Форда-Фалкерсона	31
2.11 Поток минимальной стоимости	33
2.12 Матричная теорема Кирхгофа	35
2.13 Кодирование Прюфера	36
2.14 Декодирование Прюфера	38
2.15 Алгоритм Краскала	39
2.16 Максимальное независимое множество вершин	41

2.17	Эйлеров цикл (модификация и алгоритм Флёри)	44
2.18	Фундаментальная система разрезов	47
3	Результаты работы	50
3.1	Лабораторная работа №1	50
3.2	Лабораторная работа №2	56
3.3	Лабораторная работа №3	57
3.4	Лабораторная работа №4	59
3.5	Лабораторная работа №5	62
	Заключение	65
	Список литературы	67

Введение

В настоящем отчете приводятся результаты выполнения лабораторных работ по дисциплине «Теория графов» (работы 1-5).

Задания лабораторных работ

ЛАБОРАТОРНАЯ РАБОТА № 1

1. Сформировать случайным образом связный ациклический ориентированный граф в соответствии с заданным распределением (параметры распределения задаются как константы и подбираются экспериментально) с введенным пользователем количеством вершин. Распределение брать из справочника Вадзинского: либо реализовать блок-схему, либо генерирующую формулу. Генерация происходит по степеням вершин. В зависимости от дальнейших заданий использовать ориентированный или неориентированный граф, полученный из ориентированного.
2. Посчитать эксцентриситеты вершин, выделить центр графа и диаметральные вершины.
3. Реализовать метод Шимбелла для сгенерированной весовой матрицы (пользователь вводит количество ребер). В ответе представить минимальный и/или максимальный путь (в виде матрицы). Весовая матрица генерируется в соответствии с заданным распределением. При генерации необходимо учесть генерацию только положительных значений, только отрицательных значений и смешанную (на выбор пользователя).
4. Определить возможность построения маршрута от одной заданной точки до другой (вершины вводит пользователь) и указывать количество таких маршрутов.
5. Реализовать меню.

ЛАБОРАТОРНАЯ РАБОТА № 2

1. Для заданных графов (случайно сгенерированных в предыдущей работе) реализовать один из алгоритмов:
 - с) выполнить обход вершин графа поиском в глубину.
2. Сгенерировать весовую матрицу (положительную или с отрицательными весами – выбирает пользователь) и найти кратчайший путь для двух выбранных пользователем вершин. В результате выводится не только расстояние, но и сам путь в виде последовательности вершин. Для алго-

ритма Дейкстры необходимо выводить вектор расстояний, для Флойда-Уоршалла — матрицу расстояний.

h) алгоритм Дейкстры для отрицательных весов;

3. Сравнить скорости работы реализованных алгоритмов (по количеству итераций).

ЛАБОРАТОРНАЯ РАБОТА № 3

1. На основе сгенерированного ориентированного графа получить случайным образом матрицы пропускных способностей и стоимости.
2. Для полученного графа найти максимальный поток по алгоритму Форда-Фалкерсона (или любого алгоритма из перечисленных в лекции).
3. Вычислить заданный поток минимальной стоимости (в качестве величины потока брать значение, равное $\lfloor 2/3 \cdot \max \rfloor$, где $\max$ — максимальный поток). Для этого использовать ранее реализованный алгоритм Дейкстры или Флойда-Уоршалла.

ЛАБОРАТОРНАЯ РАБОТА № 4

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) найти число остовных деревьев, используя матричную теорему Кирхгофа.
2. Построить минимальный по весу остов для сгенерированного (неориентированного) взвешенного графа, используя алгоритм:
 - a) Краскала;

Полученный остов закодировать с помощью кода Прюфера и декодировать его. Сохранять веса при кодировании обязательно.

3. Реализовать на исходном графе и полученном остове (по выбору пользователя) алгоритм:
 - d) нахождения максимального независимого множества вершин;

ЛАБОРАТОРНАЯ РАБОТА № 5

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) проверить, является ли граф эйлеровым. Если нет, то модифицировать граф (логировать, что изменено). Построить эйлеров цикл.

2. Для заданных неориентированных графов (случайно сгенерированных в первой работе) построить кратчайший остов (алгоритмом из четвертой работы). На основе данного остова получить:
- b) для четных вариантов: фундаментальную систему разрезов (вывести ее на экран), получение всех остальных разрезов на основе фундаментальных с помощью операции симметрической разности (выбранные разрезы вводит пользователь).

1 Математическое описание

1.1 Определение графа

Графом $G(V, E)$ называется совокупность двух множеств — непустого множества V (множества *вершин*) и множества E двухэлементных подмножеств множества V (E — множество *рёбер*).

$$G(V, E) \stackrel{\text{Def}}{=} \langle V; E \rangle, \quad V \neq \emptyset, \quad E \subset 2^V \text{ \& } \forall e \in E (|e| = 2). \quad (1)$$

Легко видеть, что любое множество E двухэлементных подмножеств множества V определяет симметричное бинарное отношение на множестве V . Поэтому можно считать, что

$$E \subseteq V \times V, \quad E = E^{(-1)}$$

и трактовать ребро не только как множество $\{v_1, v_2\}$, но и как пару (v_1, v_2) .

$$p \stackrel{\text{Def}}{=} p(G) \stackrel{\text{Def}}{=} |V|, \quad q \stackrel{\text{Def}}{=} q(G) \stackrel{\text{Def}}{=} |E|. \quad (2)$$

1.2 Распределение Пирсона

Для генерации случайных параметров графа используется распределение Пирсона (хи-квадрат). Формула основана на сумме квадратов нормально распределенных величин:

$$\chi_n^2 = \sum_{i=1}^n Z_i^2 \quad (3)$$

где Z_i — независимые случайные величины, распределенные по стандартному нормальному распределению. Методика генерации заимствована из справочника Вадзинского Р.Н. «Справочник по вероятностным распределениям»

На рисунке 1 показана теоритическая плотность вероятности распределения Пирсона. На рисунке 2 показана плотность вероятности распределения Пирсона, посчитанная программой. В качестве параметра распределения Пирсона выбрано $n = 3$.

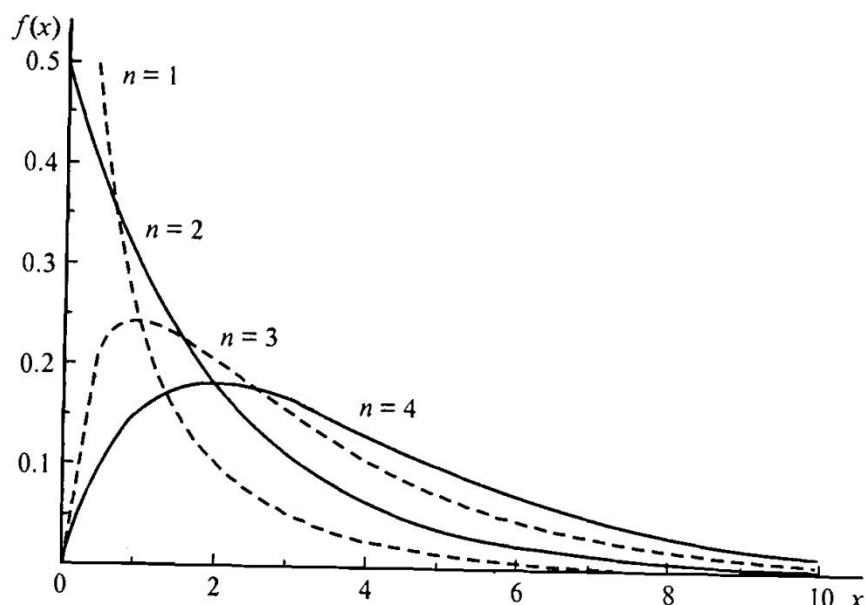


Рис. 1: Плотность распределения Пирсона (хи-квадрат) для различных степеней свободы n (из справочника Вадзинского Р.Н.)

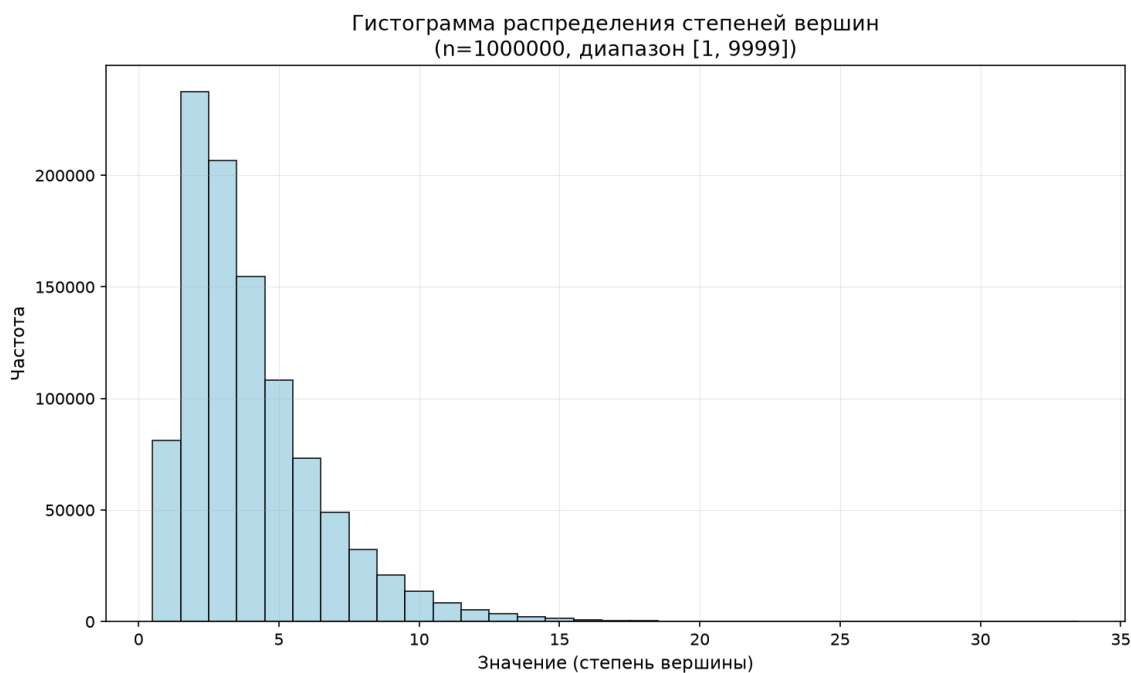


Рис. 2: Посчитанная плотность вероятности параболического распределения

1.3 Расстояния

Длиной маршрута называется количество рёбер в нём (с учётом повторений). Если маршрут $M = v_0, e_1, v_1, e_2, v_2, \dots, e_k, v_k$, то длина M равна k (обозначается $|M| = k$). *Расстоянием* между вершинами u и v (обозначается $d(u, v)$) называется длина кратчайшей цепи $\langle u, v \rangle$, а сама кратчайшая цепь называется *геодезической*:

$$d(u, v) \stackrel{\text{Def}}{=} \min_{\langle u, v \rangle} |\langle u, v \rangle|. \quad (4)$$

Если $\neg \exists \langle u, v \rangle$, то по определению $d(u, v) \stackrel{\text{Def}}{=} +\infty$.

Множество вершин, находящихся на заданном расстоянии n от вершины v (обозначение $D(v, n)$), называется *ярусом*:

$$D(v, n) \stackrel{\text{Def}}{=} \{u \in V \mid d(v, u) = n\}. \quad (5)$$

Ясно, что множество вершин V всякого связного графа однозначно разбивается на ярусы относительно данной вершины. *Диаметром* графа G называется длиннейшая геодезическая. Длина диаметра обозначается $D(G)$:

$$D(G) \stackrel{\text{Def}}{=} \max_{u, v \in V} d(u, v). \quad (6)$$

1.4 Эксцентриситет и центр

Эксцентриситетом $e(v)$ вершины v в связном графе $G(V, E)$ называется максимальное расстояние от вершины v до других вершин графа G :

$$e(v) \stackrel{\text{Def}}{=} \max_{u \in V} d(v, u). \quad (7)$$

Наиболее эксцентричные вершины — это концы диаметра.

Радиусом $R(G)$ графа G называется наименьший из эксцентриситетов вершин:

$$R(G) \stackrel{\text{Def}}{=} \min_{v \in V} e(v). \quad (8)$$

Вершина v называется *центральной*, если её эксцентриситет совпадает с радиусом графа, $e(v) = R(G)$.

Множество центральных вершин называется *центром* графа и обозначается $C(G)$:

$$C(G) \stackrel{\text{Def}}{=} \{v \in V \mid e(v) = R(G)\}. \quad (9)$$

1.5 Обходы графов

Обход графа — это некоторое систематическое перечисление его вершин (и/или рёбер). Наибольший интерес представляют обходы, использующие локальную информацию (списки смежности).

Среди всех обходов наиболее известны поиск *в ширину* и *в глубину*. Алгоритмы поиска в ширину и в глубину лежат в основе многих алгоритмов на графах.

Теорема. Если граф G связан (и конечен), то поиск в ширину и поиск в глубину обходят все вершины по одному разу.

1.6 Выявление маршрутов с заданным количеством рёбер и матрица смежности

Матрица смежности — это квадратная симметрическая матрица. Сумма элементов i -й строки равна $\deg v_i$, т. е. $\sum_{j=1}^n \alpha_{ij} = \deg v_i$. Отметим, что для обыкновенных графов матрица смежности бинарна, т. е. состоит из нулей и единиц, причем её главная диагональ целиком состоит из нулей.

$$M[i, j] = \begin{cases} 1, & \text{если вершина } v_i \text{ смежна с вершиной } v_j, \\ 0, & \text{если вершины } v_i \text{ и } v_j \text{ не смежны.} \end{cases}$$

Для матрицы смежности $n(p, q) = O(p^2)$.

Матрица смежности

- *Характеристическим многочленом графа* называется характеристический многочлен любой из его матриц смежности. Совокупность всех корней характеристического многочлена, с учетом их кратности, называется *спектром графа*.

Теорема 1.4. Пусть $A = (\alpha_{ij})_{n \times n}$ — матрица смежности графа G без петель и $A^l = (\gamma_{ij})_{n \times n}$, где $l \in \mathbb{N}$. Тогда γ_{ij} равно числу (v_i, v_j) -маршрутов длины l .

1.7 Метод Шимбелла

1. Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть

$$\begin{cases} a \cdot b = b \cdot a \Rightarrow a + b = b + a, \\ a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \end{cases} \quad (10)$$

2. Операция сложения двух величин a и b заменяется выбором из этих величин минимального (максимального) элемента, то есть

$$a + b = b + a = \min(\max)\{a, b\}, \quad (11)$$

нули при этом игнорируются. Минимальный или максимальный элемент выбирается из ненулевых элементов. Нуль в результате операции может быть получен лишь тогда, когда все элементы из выбираемых — нулевые.

$$\omega_{ij}^{(2)} = \min(\max) \left\{ \left(\omega_{i1}^{(1)} + \omega_{1j}^{(1)} \right), \left(\omega_{i2}^{(1)} + \omega_{2j}^{(1)} \right), \dots, \left(\omega_{in}^{(1)} + \omega_{nj}^{(1)} \right) \right\} \quad (12)$$

Элемент $\omega_{ij}^{(k)}$ является элементом матрицы весов путей длины k из вершины i в вершину j , полученной путём возведения исходной матрицы весов в степень k в смысле метода Шимбелла.

Сложность: $O(k \cdot n^3)$, где k — длина пути, n — количество вершин.

1.8 Обход в глубину (DFS)

Поиск в глубину — один из основных методов обхода графа, часто используемый для проверки связности, поиска циклов и топологической сортировки. Принцип работы алгоритма можно представить в формате псевдокода:

Вход: граф $G(V, E)$, представленный списками смежности Γ .

Выход: последовательность вершин обхода.

```

for  $v \in V$  do
     $x[v] := 0$                                 { вначале все вершины не отмечены }
end for
select  $v \in V$                                 { начало обхода — произвольная вершина }
 $v \rightarrow T$                                 { помещаем  $v$  в стек  $T$  (LIFO) }
 $x[v] := 1$                                     { и отмечаем вершину  $v$  }
repeat
     $u \leftarrow T$                                 { извлекаем вершину из стека  $T$  }
    yield  $u$                                 { и возвращаем её как очередную пройденную вершину }
    for  $w \in \Gamma(u)$  do
        if  $x[w] = 0$  then
             $w \rightarrow T$                                 { помещаем  $w$  в стек  $T$  }
             $x[w] := 1$                                 { и отмечаем вершину  $w$  }
        end if
    end for
until  $T = \emptyset$ 

```

Структура данных T в данном алгоритме является *стеком* (LIFO — Last In First Out): последняя добавленная вершина извлекается первой. Это соответствует обходу графа *в глубину* (DFS). Если вместо стека использовать *очередь* (FIFO — First In First Out), тот же алгоритм будет реализовывать обход *в ширину* (BFS).

Временная сложность алгоритма составляет $O(V + E)$, где V — количество вершин, а E — количество ребер.

1.9 Алгоритм Дейкстры

Если в алгоритме поиска в ширину заменить очередь «первым вошел, первым вышел» (FIFO) на очередь с приоритетами, где ключом вершины является ее пробное расстояние $\text{dist}(v)$, то получим классический алгоритм Дейкстры. Алгоритм находит кратчайший путь от одной вершины до всех остальных во взвешенном графе. Принцип его работы можно представить в виде псевдокода:

```
Dijkstra( $s$ ):  
  InitSSSP( $s$ )  
  Insert( $s, 0$ )  
  while приоритетная очередь не пуста do  
     $u \leftarrow \text{ExtractMin}()$   
    for all рёбер  $u \rightarrow v$  do  
      if  $u \rightarrow v$  является напряжённым then  
        Relax( $u \rightarrow v$ )  
        if  $v$  находится в приоритетной очереди then  
          DecreaseKey( $v, \text{dist}(v)$ )  
        else  
          Insert( $v, \text{dist}(v)$ )  
      end if  
    end for  
  end while
```

Алгоритм Дейкстры выполняет не более E операций **DecreaseKey** и не более V операций **Insert** и **ExtractMin**. Таким образом, если мы реализуем внутреннюю очередь с приоритетами с использованием стандартной двоичной кучи, которая поддерживает выполнение каждой операции за время $O(\log V)$, то алгоритм Дейкстры выполняется за время $O(E \log V)$. К сожалению, когда исходный граф содержит отрицательные рёбра, одна и та же вершина может быть извлечена (**Extract**) несколько раз, напряжение одного и того же ребра может быть ослаблено несколько раз, а расстояния, возможно, не будут просматриваться в порядке возрастания. Для графов без отрицательных циклов, но и без других ограничений по весам рёбер время выполнения алгоритма Дейкстры в наихудшем случае является экспоненциальным — $O(2^V)$.

1.10 Поток в сети

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое *потоком* по этой дуге e , для которой выполняются свойства:

1. для каждой дуги $e \in E$ верно $f(e) \leq c(e)$, т.е. поток по любой дуге не превышает пропускную способность этой дуги;
2. для каждой вершины $v \in V$, $v \neq s, t$, верно

$$D_f(v) = \sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) = 0, \quad (13)$$

т.е. поток, входящий в любую вершину, кроме источника и стока, без остатка выходит из неё (сохранение потока).

При этом неотрицательное число

$$p_f = \sum_{e \in A(s)} f(e) \quad (14)$$

называется **величиной потока** f .

Число $w(f) \stackrel{\text{Def}}{=} \text{div}(f, s)$ называется величиной потока f . Если $f(u, v) = c(u, v)$, то дуга (u, v) называется *насыщенной*.

1.11 Алгоритм Форда-Фалкерсона

Алгоритм Форда-Фалкерсона решает задачу нахождения максимального потока в сети. Основная идея заключается в итеративном увеличении величины потока путем поиска увеличивающих путей от источника к стоку. Принцип работы алгоритма можно представить в виде псевдокода:

Вход: сеть $G(V, E)$ с источником s и стоком t , заданная матрица пропускных способностей C : `array [1..p, 1..p] of real`.

Выход: матрица максимального потока F : `array [1..p, 1..p] of real`.

```

for  $u, v \in V$  do
     $F[u, v] := 0$                                 { вначале поток нулевой }
end for
 $M:$                                               { итерация увеличения потока }
for  $v \in V$  do
     $S[v] := 0; N[v] := 0; P[v] := (+, \text{nil}, 0)$     { инициализация }
end for
 $S[s] := 1; P[s] := (+, \text{nil}, \infty)$            { так как  $s \in S$  }
repeat
     $a := 0$                                        { признак расширения  $S$  }
    for  $v \in V$  do
        if  $S[v] = 1 \ \& \ N[v] = 0$  then
            for  $u \in \Gamma(v)$  do
                if  $S[u] = 0 \ \& \ F[v, u] < C[v, u]$  then

```

1

```

        S[u] := 1; P[u] := (+, v, min(P[v].δ, C[v, u] - F[v, u])); a :=
    end if
end for
for u ∈ Γ-1(v) do
    if S[u] = 0 & F[u, v] > 0 then
        S[u] := 1; P[u] := (-, v, min(P[v].δ, F[u, v])); a := 1
    end if
end for
N[v] := 1 { узел v отмечен }
end if
end for
if S[t] then
    x := t { текущий узел аугментальной цепи }
    δ := P[t].δ { величина изменения потока }
    while x ≠ s do
        if P[x].s = + then
            F[P[x].n, x] := F[P[x].n, x] + δ { увеличение потока }
        else
            F[x, P[x].n] := F[x, P[x].n] - δ { увеличение потока }
        end if
        x := P[x].n { предыдущий узел аугментальной цепи }
    end while
    goto M
end if
until a = 0

```

Алгоритм не конкретизирует способ поиска пути, поэтому гарантированно сходится только для целочисленных пропускных способностей. При больших значениях пропускных способностей может работать длительное время. **Сложность:** $O(F \cdot (V \cdot E))$, где F — величина потока, V — количество вершин, E — количество рёбер.

1.12 Задача нахождения заданного потока минимальной стоимости

Рассмотрим задачу определения потока заданной величины θ от s к t в сети $G = (S, U, \Omega, D)$, в которой каждая дуга (x_i, x_j) характеризуется пропускной способностью $d(x_i, x_j) \in \Omega$ и неотрицательной стоимостью $d(x_i, x_j) \in D$ пересылки единичного потока из x_i в x_j вдоль дуги (x_i, x_j) .

Если $\theta > \varphi_{\max}$, где $\varphi_{\max}$ — величина максимального потока в сети G от s к t , то решения нет. Если же $\theta \leq \varphi_{\max}$, то может быть определено

несколько различных потоков величины θ от s к t . Математическая модель задачи выглядит следующим образом. Минимизировать

$$\sum_{(x_i, x_j) \in U} d(x_i, x_j) \cdot \varphi(x_i, x_j), \quad (15)$$

где $\varphi(x_i, x_j)$ — поток по дуге (x_i, x_j) при ограничениях:

1. $0 \leq \varphi(x_i, x_j) \leq d(x_i, x_j) \exists (x_i, x_j) \in U$;
2. для начальной вершины $s \in S$: $\sum_{x_j \in S} \varphi(s, x_j) - \sum_{x_j \in S} \varphi(x_j, s) = \theta$ — уравнение истока;
3. для конечной вершины $t \in S$: $\sum_{x_j \in S} \varphi(t, x_j) - \sum_{x_j \in S} \varphi(x_j, t) = -\theta$ — уравнение стока;
4. для любой другой вершины $x_j \neq s, t$: $\sum_{x_i \in S} \varphi(x_j, x_i) - \sum_{x_i \in S} \varphi(x_i, x_j) = 0$ — условие сохранения потока.

Модификация сети допускается только определённого порядка. Пусть $G = (S, U, \Omega, D)$ — исходная сеть, S — множество вершин, U — множество рёбер, Ω — множество весов (пропускных способностей дуг), D — набор стоимостей пересылки единичного потока из x_i в x_j вдоль дуги (x_i, x_j) .

Модифицированная относительно данного потока φ сеть $\tilde{G} = (\tilde{S}, \tilde{U}, \tilde{\Omega}, \tilde{D})$ строится следующим образом.

1. $\tilde{S} = S$, то есть число и набор вершин в модифицированной сети не изменяется.
2. $\tilde{U} = U \cup V$, где V — некоторое множество фиктивных дуг. Таким образом, после модификации число дуг сети увеличивается.
3. Если $(x_i, x_j) \in S$ и $\varphi(x_i, x_j) > 0$, то дуга (x_j, x_i) включается в множество V . При этом полагается $\tilde{c}(x_j, x_i) = \varphi(x_i, x_j)$, $\tilde{d}(x_j, x_i) = -d(x_i, x_j)$. Этот пункт применяется только к дугам, по которым проходит поток φ , относительно которого происходит модификация сети.
4. Для всех ненасыщенных дуг, где нет противоположного потока, в том числе и для ненасыщенных дуг с потоком, относительно которого происходит модификация сети, то есть для $(x_i, x_j) \in S$, $\varphi(x_i, x_j) < d(x_i, x_j)$ и $\varphi(x_j, x_i) = 0$ полагают $\tilde{c}(x_i, x_j) = d(x_i, x_j) - \varphi(x_i, x_j)$, $\tilde{d}(x_i, x_j) = d(x_i, x_j)$.
5. Для всех насыщенных дуг $(x_i, x_j) \in S$, $\varphi(x_i, x_j) = d(x_i, x_j)$ и $\varphi(x_j, x_i) = 0$ полагают $\tilde{c}(x_i, x_j) = 0$, $\tilde{d}(x_i, x_j) = \infty$.

Сложность: $O(F \cdot (V \cdot E))$, где F — величина потока, V — количество вершин, E — количество рёбер.

1.13 Матрица Кирхгофа

Определим *матрицу Кирхгофа* $B = B(G) = (\beta_{ij})_{n \times n}$, полагая

$$B(G) = \begin{pmatrix} \deg v_1 & 0 & \cdots & 0 \\ 0 & \deg v_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \deg v_n \end{pmatrix} - A(G), \quad (16)$$

где $A(G)$ — матрица смежности графа G . Иными словами,

$$\beta_{ij} = \begin{cases} -1, & \text{если } i \neq j \text{ и } v_i \text{ смежна с } v_j, \\ 0, & \text{если } i \neq j \text{ и } v_i \text{ не смежна с } v_j, \\ \deg v_i, & \text{если } i = j. \end{cases} \quad (17)$$

Отметим, что обыкновенный граф G может иметь несколько различных матриц Кирхгофа, отвечающих различным упорядочениям графа G , и все эти матрицы подобны между собой.

Лемма 1. *Алгебраические дополнения всех элементов матрицы Кирхгофа равны между собой.*

Теорема Кирхгофа. *Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа $B(G)$.*

Все алгебраические дополнения матрицы Кирхгофа равны между собой.

Т. Кирхгофа позволяет найти число остовных деревьев в связном графе со сложностью алгоритма $O(n^3)$

1.14 Кратчайший остов

Пусть $G(V, E)$ — граф. *Остовный подграф* графа $G(V, E)$ — это подграф, содержащий все вершины. Остовный подграф, являющийся деревом, называется **остовом** или **каркасом**.

Остов определяется *множеством рёбер*, поскольку вершины остова суть вершины графа.

Несвязный граф *не имеет остова*. Связный граф может иметь много остовов.

Если задать длины рёбер, то можно поставить задачу нахождения **кратчайшего остова**.

1.15 Алгоритм Краскала

Алгоритм Краскала строит минимальное остовное дерево путем жадного выбора рёбер с наименьшим весом, которые не создают цикл.

Входные данные: список E всех ребер графа G с длинами (весами), упорядоченный в порядке возрастания.

Выходные данные: множество T ребер, образующих минимальное остовное дерево.

```

 $T := \emptyset$ 
 $k := 1$ 
for  $i := 1$  to  $p - 1$  do
    while  $T \cup \{E[k]\}$  содержит цикл do
         $k := k + 1$ 
    end while
     $T := T \cup \{E[k]\}$ 
     $k := k + 1$ 
end for

```

По завершении алгоритма множество T содержит ровно $p - 1$ ребер и образует минимальное остовное дерево.

Сложность: $O(E \log E)$, где E — количество рёбер.

1.16 Код Прюфера

Код Прюфера — наиболее экономное по памяти представление дерева. Вершины дерева $T(V, E)$ занумерованы числами из интервала $1..p$.

Построение последовательности A : **array** $[1..p - 1]$ **of** $1..p$.

```

for  $i := 1$  to  $p - 1$  do do
     $v := \min\{k \in V \mid d(k) = 1\}$ 
     $A[i] := \Gamma(v)$ 
     $V := V \setminus \{v\}$ 
end for

```

На каждом шаге из дерева удаляется висячая вершина с наименьшим номером, а номер её единственного соседа записывается в код. Процесс повторяется до тех пор, пока в дереве не останется две вершины. Полученная последовательность длины $p - 2$ (или $p - 1$ с учётом последнего ребра) является кодом Прюфера.

Восстановление дерева по коду Прюфера выполняется следующим образом.

```

 $E := \emptyset$ 
 $B := \{1, 2, \dots, p\}$ 
for  $i := 1$  to  $p - 2$  do do
     $v := \min\{k \in B \mid k \text{ не встречается в } A[i..p - 2]\}$ 
     $E := E \cup \{(v, A[i])\}$ 
     $B := B \setminus \{v\}$ 
end for

```

Сначала создаётся массив степеней вершин, изначально заполненный единицами. Для каждой вершины, встречающейся в коде, её степень увеличивается на единицу, поскольку каждое появление вершины в коде означает, что она была соседом удалённой висячей вершины.

Далее на каждом шаге выбирается вершина v с наименьшим номером, имеющая степень 1 (т. е. висячая вершина), и добавляется ребро $(v, A[i])$, где $A[i]$ — очередной элемент кода. Затем степени выбранной вершины и её соседа уменьшаются на единицу. Процесс повторяется для всех элементов кода.

После обработки всех элементов кода остаются две вершины со степенью 1 — они соединяются ребром, завершая построение дерева.

Сложность: $O(n \log n)$, где n — количество вершин.

1.17 Нахождение максимального независимого множества вершин

Множество вершин называется *независимым* (или *внутренне устойчивым*), если никакие две из них не смежны.

Для решения задачи о нахождении максимального независимого множества вершин в исходном графе выбран алгоритм построения максимальных независимых множеств вершин, основанный на переборе с возвратом. Далее алгоритм представлен в виде псевдокода.

Вход: граф $G(V, E)$, заданный списками смежности $\Gamma[v]$.

Выход: последовательность максимальных независимых множеств.

```

 $k := 0$  { количество элементов в текущем независимом множестве }
 $S[k] := \emptyset$  {  $S[k]$  — независимое множество из  $k$  вершин }
 $Q^-[k] := \emptyset$  {  $Q^-[k]$  — множество вершин, уже использованных для
расширения  $S[k]$  }
 $Q^+[k] := V$  {  $Q^+[k]$  — множество вершин, которые можно использовать для
расширения  $S[k]$  }
M1: { шаг вперёд }
select  $v \in Q^+[k]$  { расширяющая вершина }
 $S[k+1] := S[k] \cup \{v\}$  { расширенное множество }
 $Q^-[k+1] := Q^-[k] \cup \Gamma[v]$  { вершина  $v$  использована для расширения }
 $Q^+[k+1] := Q^+[k] \setminus (\Gamma[v] \cup \{v\})$  { все вершины, смежные с  $v$ , не могут быть
использованы для расширения }
 $k := k + 1$ 
M2: { проверка }
for  $u \in Q^-[k]$  do
  if  $\Gamma[u] \cap Q^+[k] = \emptyset$  then
    goto M3 { можно возвращаться }
  end if

```

```

end for
if  $Q^+[k] = \emptyset$  then
    if  $Q^-[k] = \emptyset$  then
        yield  $S[k]$                                 { множество  $S[k]$  максимально }
    end if
    goto M3                                          { можно возвращаться }
else
    goto M1                                          { можно идти вперёд }
end if
M3:                                                  { шаг назад }
 $v := \text{last}(S[k])$                                 { последний добавленный элемент }
 $k := k - 1$ 
 $S[k] := S[k + 1] \setminus \{v\}$ 
 $Q^-[k] := Q^-[k] \cup \{v\}$                         { вершина  $v$  уже добавлялась }
 $Q^+[k] := Q^+[k] \setminus \{v\}$ 
if  $k = 0$  &  $Q^+[k] = \emptyset$  then
    stop                                            { перебор завершён }
else
    goto M2                                          { переход на проверку }
end if

```

Сложность алгоритма: в наихудшем случае алгоритм перебирает все подмножества вершин, поэтому его сложность оценивается как $O(2^n)$, где n — количество вершин графа. На практике за счёт отсечений (проверка $Q^-[k]$ и $Q^+[k]$) число рассматриваемых ветвей существенно сокращается.

1.18 Эйлеровы циклы и графы

Если граф имеет цикл (не обязательно простой), содержащий все рёбра графа, то такой цикл называется **эйлеровым циклом**, а граф называется **эйлеровым графом**.

Если граф имеет цепь (не обязательно простую), содержащую все рёбра, то такая цепь называется *эйлеровой цепью*, а граф называется *полуэйлеровым графом*.

Эйлеров цикл содержит не только все рёбра (по одному разу), но и все вершины графа (возможно, по несколько раз).

Эйлеровым может быть только связный граф.

ТЕОРЕМА Эйлера. *Если граф G связан и нетривиален, то следующие утверждения эквивалентны:*

1. G — эйлеров граф.
2. Каждая вершина G имеет чётную степень.

3. Множество рёбер G можно разбить на простые циклы.

Алгоритм построения эйлерова цикла в эйлеровом графе (Флери).

Вход: эйлеров граф $G(V, E)$, заданный списками смежности ($\Gamma[v]$ — список вершин, смежных с вершиной v).

Выход: последовательность вершин эйлерова цикла.

```

 $S := \emptyset$  { стек для хранения вершин }
select  $v \in V$  { произвольная вершина }
 $v \rightarrow S$  { положить  $v$  в стек  $S$  }
while  $S \neq \emptyset$  do
     $v := \text{top } S$  {  $v$  — верхний элемент стека }
    if  $\Gamma[v] = \emptyset$  then
         $v \leftarrow S$ ; yield  $v$  { очередная вершина эйлерова цикла }
    else
        select  $u \in \Gamma[v]$  { взять первую вершину из списка смежности }
         $u \rightarrow S$  { положить  $u$  в стек }
         $\Gamma[v] := \Gamma[v] - u$ ;  $\Gamma[u] := \Gamma[u] - v$  { удалить ребро  $(v, u)$  }
    end if
end while

```

Если реализовать поиск рёбер, инцидентных вершине, и удаление рёбер за $O(1)$, то алгоритм будет работать за $O(p)$.

1.19 Фундаментальная система разрезов

Пусть опять $T(V, E_T)$ — некоторый остов графа $G(V, E)$. Рассмотрим ребро остова e и определим разрез S_e следующим образом.

Ребро e — *мост* в дереве T . Следовательно, удаление ребра e разбивает множество вершин V на два непустых подмножества V_1 и V_2 , так что

$$V_1 \subset V, V_1 \neq \emptyset, V_2 \subset V, V_2 \neq \emptyset, V_1 \cup V_2 = V, V_1 \cap V_2 = \emptyset.$$

Включим в разрез S_e ребро e и все хорды остова T , соединяющие вершины множества V_1 с вершинами множества V_2 (Тогда S_e — это простой разрез):

$$S_e \stackrel{\text{Def}}{=} e + \{(v_1, v_2) \in T^* \mid v_1 \in V_1 \text{ \& } v_2 \in V_2\} = E(V_1, V_2). \quad (18)$$

Система разрезов называется **фундаментальной системой разрезов**.

Разрезы фундаментальной системы называются *фундаментальными*, а количество разрезов в (данной) фундаментальной системе называется *коциклическим рангом* (или *коцикломатическим числом*) графа G и обозначается $m^*(G)$.

$$S \stackrel{\text{Def}}{=} \{S_e\}_{e \in T} \quad (19)$$

2 Особенности реализации программы

2.1 Структура данных MyGraph

Основной структурой программы является `MyGraph`, которая содержит все необходимые представления графа и результатов вычислений:

```
1 struct MyGraph {
2     vector<vector<int>> adjMatrix;           // матрица смежности
3     vector<vector<int>> distMatrix;         // матрица расстояний
4     vector<vector<int>> weightMatrix;       // матрица весов
5     vector<vector<int>> costMatrix;         // матрица стоимостей
6     vector<vector<int>> capacityMatrix;     // матрица пропускных
    ↪     способностей
7     vector<vector<int>> flowMatrix;         // матрица потоков
8     vector<vector<int>> ostAdjMatrix;       // матрица смежности остова
9     vector<vector<int>> ostWeightMatrix;   // матрица весов остова
10    vector<int> pruferCode;                // код Прюфера
11    int verticesCount;
12    bool isGenerated;
13    bool hasWeights;
14    bool hasCosts;
15    bool hasCapacities;
16    bool isDirected;
17    int lastMaxFlow;
18    int lastSource;
19    int lastSink;
20 };
```

Листинг 1: Структура `MyGraph`

2.2 Генератор случайных чисел

Класс `RandomGenerator` обеспечивает генерацию случайных чисел с нормальным распределением для реализации распределения Пирсона.

```
1 class RandomGenerator {
2 private:
3     mt19937 gen;
4     normal_distribution<> normalDistrubution;
5     uniform_int_distribution<int> boolDist;
6
7 public:
8     RandomGenerator() : gen(random_device{}()),
9                        normalDistrubution(0.0, 1),
10                        boolDist(0, 1) {}
11 }
```

```

12     double getNormal() {
13         return normalDistrubution(gen);
14     }
15
16     bool getBool() {
17         return boolDist(gen);
18     }
19 };
20
21 double PirsonDistribution(int n, RandomGenerator& rng) {
22     double sum = 0.0;
23     for (int i = 0; i < n; i++) {
24         double u = rng.getNormal();
25         sum += u * u;
26     }
27     return sum;
28 }

```

Листинг 2: Класс генератора и функция распределения Пирсона

2.3 Генерация степеней вершин

Функция генерирует степени вершин в соответствии с распределением Пирсона. Для неориентированных графов гарантируется четная сумма степеней (необходимое условие реализуемости графа)

Вход: количество вершин n , объект генератора случайных чисел, флаг направленности

Выход: вектор степеней вершин

```

1 vector<int> generateDegrees(int n, RandomGenerator& rng, bool directed) {
2     if (n == 0) return vector<int>();
3     if (n == 1) {
4         vector<int> degrees(1, 0);
5         return degrees;
6     }
7
8     vector<int> degrees(n);
9
10    for (int i = 0; i < n; i++) {
11        double x = PirsonDistribution(PARAM, rng);
12        degrees[i] = (int)round(x) + 1;
13        degrees[i] = max(1, min(degrees[i], n - 1));
14    }
15
16    if (!directed) {
17        int sumDeg = accumulate(degrees.begin(), degrees.end(), 0);
18        if (sumDeg % 2 != 0) {
19            degrees[0] = min(degrees[0] + 1, n - 1);

```

```

20     }
21 }
22
23     return degrees;
24 }

```

Листинг 3: Генерация степеней вершин

2.4 Построение графа

Два вспомогательных алгоритма для построения ориентированного и неориентированного графов по вектору степеней. Функция `buildDirected` создает ориентированный граф с заданными исходящими степенями, а `buildUndirected` создает неориентированный граф путем симметризации.

Входные данные: вектор степеней, матрица смежности, генератор случайных чисел

Выходные данные: заполненная матрица смежности

```

1 bool buildDirected(const vector<int>& outDegrees, vector<vector<int>>&
  ↪ adjMatrix, RandomGenerator& rng) {
2     int n = outDegrees.size();
3     if (n == 0) return true;
4     if (n == 1) {
5         if (outDegrees[0] == 0) {
6             adjMatrix[0][0] = 0;
7             return true;
8         }
9         return false;
10    }
11
12    for (int i = 0; i < n; i++) {
13        fill(adjMatrix[i].begin(), adjMatrix[i].end(), 0);
14    }
15
16    vector<int> remainingOut = outDegrees;
17
18    for (int i = 0; i < n; i++) {
19        int need = remainingOut[i];
20        if (need == 0) continue;
21
22        vector<int> availablePositions;
23        for (int j = i + 1; j < n; j++) {
24            availablePositions.push_back(j);
25        }
26
27        while (need > 0 && !availablePositions.empty()) {
28            int randomIndex = (static_cast<int>(round(PirsonDistribution(5,
  ↪ rng)))) % availablePositions.size();

```

```

29         int position = availablePositions[randomIndex];
30
31         adjMatrix[i][position] = 1;
32
33         availablePositions[randomIndex] = availablePositions.back();
34         availablePositions.pop_back();
35
36         need--;
37     }
38 }
39
40 return true;
41 }
42
43 bool buildUndirected(const vector<int>& degrees, vector<vector<int>>&
↪ adjMatrix, RandomGenerator& rng) {
44     int n = degrees.size();
45     buildDirected(degrees, adjMatrix, rng);
46
47     for (int i = 0; i < n; i++) {
48         for (int j = i + 1; j < n; j++) {
49             if (adjMatrix[i][j] == 1) {
50                 adjMatrix[j][i] = 1;
51             }
52         }
53     }
54
55     return true;
56 }

```

Листинг 4: Построение ориентированного и неориентированного графов

2.5 Поиск эксцентриситета и центра графа

Алгоритм Флойда-Уоршалла вычисляет матрицу расстояний между всеми парами вершин, затем для каждой вершины определяется эксцентриситет (максимальное расстояние до других вершин). Центр — множество вершин с минимальным эксцентриситетом (= радиус), диаметральные вершины — с максимальным (= диаметр).

Входные данные: матрица смежности графа

Выходные данные: эксцентриситеты вершин, центр графа, диаметр, радиус, диаметральные вершины

```

1 void calculateEccentricity(MyGraph& graph) {
2     if (!graph.isGenerated) {
3         cout << "сначала сгенерируйте граф\n";
4         return;
5     }

```

```

6
7     int n = graph.verticesCount;
8
9     if (n <= 1) {
10         if (n == 0) {
11             cout << "пустой граф\n";
12             return;
13         }
14         cout << "эксцентриситет вершины 0: 0\n";
15         cout << "\nрадиус графа: 0\n";
16         cout << "диаметр графа: 0\n";
17         cout << "центр графа: 0\n";
18         cout << "диаметральные вершины: 0\n";
19         return;
20     }
21
22     // инициализация матрицы расстояний
23     for (int i = 0; i < n; i++) {
24         for (int j = 0; j < n; j++) {
25             if (i == j) {
26                 graph.distMatrix[i][j] = 0;
27             } else if (graph.adjMatrix[i][j] == 1) {
28                 graph.distMatrix[i][j] = 1;
29             } else {
30                 graph.distMatrix[i][j] = INF;
31             }
32         }
33     }
34
35     for (int k = 0; k < n; k++) {
36         for (int i = 0; i < n; i++) {
37             for (int j = 0; j < n; j++) {
38                 if (graph.distMatrix[i][k] < INF && graph.distMatrix[k][j] <
39                     → INF) {
40                     if (graph.distMatrix[i][j] > graph.distMatrix[i][k] +
41                         → graph.distMatrix[k][j]) {
42                         graph.distMatrix[i][j] = graph.distMatrix[i][k] +
43                             → graph.distMatrix[k][j];
44                     }
45                 }
46             }
47         }
48     }
49
50     // вычисление эксцентриситетов
51     vector<int> eccentricity(n, 0);
52     int radius = INF;
53     int diameter = 0;
54
55     for (int i = 0; i < n; i++) {
56         int maxDist = 0;
57         for (int j = 0; j < n; j++) {

```

```

55         if (graph.distMatrix[i][j] > maxDist && graph.distMatrix[i][j] <
            ↪ INF) {
56             maxDist = graph.distMatrix[i][j];
57         }
58     }
59     eccentricity[i] = maxDist;
60
61     if (maxDist < radius) radius = maxDist;
62     if (maxDist > diameter) diameter = maxDist;
63
64     cout << "эксцентриситет вершины " << i << ": " << eccentricity[i] <<
        ↪ endl;
65 }
66
67 cout << "\nрадиус графа: " << radius << endl;
68 cout << "диаметр графа: " << diameter << endl;
69
70 cout << "центр графа (вершины с эксцентриситетом = радиусу): ";
71 for (int i = 0; i < n; i++) {
72     if (eccentricity[i] == radius) {
73         cout << i << " ";
74     }
75 }
76 cout << endl;
77
78 cout << "диаметральные вершины (эксцентриситет = диаметру): ";
79 for (int i = 0; i < n; i++) {
80     if (eccentricity[i] == diameter) {
81         cout << i << " ";
82     }
83 }
84 cout << endl;
85 }

```

Листинг 5: Вычисление эксцентриситета и центра графа

2.6 Метод Шимбелла

Метод Шимбелла находит матрицы минимальных и максимальных путей длиной ровно k рёбер. Вспомогательная функция `findPathsOfLength` итеративно умножает матрицу весов сама на себя $k - 1$ раз с заменой операций: вместо сложения и умножения используются $\min / \max$ и суммирование. Если рёбра нет (вес 0), ячейка содержит `INF_VAL = INT_MAX`, что означает «нет пути».

Входные данные: весовая матрица, длина пути k , тип поиска (минимум/-максимум)

Выходные данные: матрица минимальных и матрица максимальных путей длины ровно k рёбер

```

1 vector<vector<int>> findPathsOfLength(const vector<vector<int>>& weightMatrix,
2                                     int pathLength, int searchType) {
3     int n = weightMatrix.size();
4     const int INF_VAL = std::numeric_limits<int>::max();
5
6     // длина 1 - копия матрицы весов, но 0 (нет ребра) заменяется на INF
7     vector<vector<int>> current(n, vector<int>(n, INF_VAL));
8     for (int i = 0; i < n; i++)
9         for (int j = 0; j < n; j++)
10             if (weightMatrix[i][j] != 0)
11                 current[i][j] = weightMatrix[i][j];
12
13     if (pathLength == 1) return current;
14
15     int initValue = (searchType == 1) ? INF_VAL :
16         ↪ std::numeric_limits<int>::min();
17
18     for (int p = 1; p < pathLength; p++) {
19         vector<vector<int>> next(n, vector<int>(n, INF_VAL));
20         for (int i = 0; i < n; i++) {
21             for (int j = 0; j < n; j++) {
22                 bool found = false;
23                 int bestValue = initValue;
24                 for (int k = 0; k < n; k++) {
25                     if (current[i][k] != INF_VAL && weightMatrix[k][j] != 0) {
26                         found = true;
27                         int pathWeight = current[i][k] + weightMatrix[k][j];
28                         if (searchType == 1) {
29                             if (pathWeight < bestValue) bestValue =
30                                 ↪ pathWeight;
31                         } else {
32                             if (pathWeight > bestValue) bestValue =
33                                 ↪ pathWeight;
34                         }
35                     }
36                 }
37                 if (found) next[i][j] = bestValue;
38             }
39         }
40         current = next;
41     }
42     return current;
43 }
44
45 void shimbellMethod(MyGraph& graph) {
46     int n = graph.verticesCount();
47     if (n <= 1) { if (n == 0) cout << "пустой граф\n"; return; }
48     if (!graph.hasWeights) {
49         cout << "требуется сгенерировать весовую матрицу"; return;
50     }
51
52     int pathLength;

```

```

50     cout << "Введите длину пути (количество ребер): ";
51     cin >> pathLength;
52
53     if (pathLength == 0) {
54         vector<vector<int>> zeroLenMatrix(n, vector<int>(n,
55             ↪ std::numeric_limits<int>::max()));
56         for (int i = 0; i < n; i++) zeroLenMatrix[i][i] = 0;
57         printMatrix(zeroLenMatrix, "матрица минимальных путей");
58         printMatrix(zeroLenMatrix, "матрица максимальных путей");
59         return;
60     }
61
62     if (pathLength < 0 || pathLength > n - 1) {
63         cout << "длина пути должна быть положительной и не превышать n-1\n";
64         return;
65     }
66
67     vector<vector<int>> minResult = findPathsOfLength(graph.weightMatrix,
68         ↪ pathLength, 1);
69     vector<vector<int>> maxResult = findPathsOfLength(graph.weightMatrix,
70         ↪ pathLength, 2);
71     printMatrix(minResult, "матрица минимальных путей");
72     printMatrix(maxResult, "матрица максимальных путей");
73 }

```

Листинг 6: Метод Шимбелла

2.7 Подсчет маршрутов

Алгоритм подсчитывает количество маршрутов между двумя вершинами, суммируя степени матрицы смежности от 0 до $n - 1$. После суммирования нули (нет маршрутов) заменяются на `INT_MAX`, чтобы функция `printMatrix` выводила «-» вместо нуля.

Входные данные: матрица смежности, пара вершин

Выходные данные: матрица сумм маршрутов всех длин от 0 до $n - 1$ и количество маршрутов между заданной парой

```

1 void countRoutes(MyGraph& graph) {
2     if (!graph.isGenerated) { cout << "сначала сгенерируйте граф\n"; return; }
3     int n = graph.verticesCount;
4     if (n == 0) { cout << "пустой граф\n"; return; }
5
6     int start, end;
7     cout << "Введите начальную вершину: "; cin >> start;
8     cout << "Введите конечную вершину: "; cin >> end;
9     if (start < 0 || start >= n || end < 0 || end >= n) {
10         cout << "неверный номер вершины\n"; return;
11     }

```

```

12
13     if (n == 1) {
14         if (start == 0 && end == 0)
15             cout << "общее количество маршрутов (включая путь длины 0): 1\n";
16         else
17             cout << "маршрут не существует\n";
18         return;
19     }
20
21     // сумма матриц смежности в степенях от 0 до n-1
22     vector<vector<int>> sum(n, vector<int>(n, 0));
23     for (int i = 0; i < n; i++) sum[i][i] = 1;
24
25     for (int p = 1; p < n; p++) {
26         vector<vector<int>> power = matrixPower(graph.adjMatrix, p);
27         for (int i = 0; i < n; i++)
28             for (int j = 0; j < n; j++)
29                 sum[i][j] += power[i][j];
30     }
31
32     // нет маршрутов -> INT_MAX, чтобы printMatrix отрисовал "-"
33     const int INF_VAL = std::numeric_limits<int>::max();
34     for (int i = 0; i < n; i++)
35         for (int j = 0; j < n; j++)
36             if (sum[i][j] == 0) sum[i][j] = INF_VAL;
37
38     printMatrix(sum, "матрица сумм маршрутов (длины от 0 до " + to_string(n -
39         ↪ 1) + ")");
40
41     cout << "\nобщее количество маршрутов из " << start << " в " << end << ":
42     ↪ ";
43     if (sum[start][end] == INF_VAL) cout << "- (маршрутов не существует)\n";
44     else cout << sum[start][end] << "\n";
45 }

```

Листинг 7: Подсчет маршрутов

2.8 Поиск в глубину (DFS)

Итеративный обход графа в глубину на основе стека (LIFO).

Входные данные: матрица смежности, начальная вершина

Выходные данные: последовательность посещённых вершин

```

1 void depthFirstSearch(const MyGraph& graph) {
2     if (!graph.isGenerated) { cout << "сначала сгенерируйте граф\n"; return; }
3
4     int iterations = 0;
5     int n = graph.verticesCount;
6     if (n == 0) { cout << "пустой граф\n"; return; }

```

```

7
8     int startVertex;
9     cout << "введите начальную вершину: ";
10    cin >> startVertex;
11    if (startVertex < 0 || startVertex >= n) {
12        cout << "неверный номер вершины\n"; return;
13    }
14
15    vector<int> visited(n, 0);
16    stack<int> st;
17    st.push(startVertex);
18    visited[startVertex] = 1;
19
20    cout << "\нобход графа в глубину (dfs):\n";
21    cout << "последовательность вершин: ";
22
23    while (!st.empty()) {
24        int u = st.top();
25        st.pop();
26        cout << u << " ";
27
28        for (int w = n - 1; w >= 0; w--) {
29            if (graph.adjMatrix[u][w] == 1) {
30                if (visited[w] == 0) {
31                    st.push(w);
32                    visited[w] = 1;
33                }
34            }
35            iterations++;
36        }
37    }
38    cout << endl << "итераций совершено: " << iterations << endl;
39 }

```

Листинг 8: Поиск в глубину

2.9 Алгоритм Дейкстры

Алгоритм находит кратчайшие пути от одной вершины до всех остальных. Использует приоритетную очередь `priority_queue` для извлечения вершины с минимальным расстоянием.

Входные данные: матрица смежности, матрица весов, исходная вершина

Выходные данные: вектор расстояний `dist`, вектор предшественников `prev`

```

1 pair<vector<int>, vector<int>> dijkstra(int n, int source,
2                                     const vector<vector<int>>& adjMatrix,
3                                     const vector<vector<int>>&
4                                     ↪ weightMatrix) {

```

```

4    vector<int> dist(n, INF);
5    vector<int> prev(n, -1);
6
7    using pii = pair<int, int>;
8    priority_queue<pii, vector<pii>, greater<pii>> pq;
9
10   long long iterations = 0;
11
12   dist[source] = 0;
13   pq.push({0, source});
14
15   while (!pq.empty()) {
16       auto [d, u] = pq.top();
17       pq.pop();
18
19       for (int v = 0; v < n; v++) {
20           iterations++;
21           if (adjMatrix[u][v] == 1) {
22               int w = weightMatrix[u][v];
23               if (dist[u] + w < dist[v]) {
24                   dist[v] = dist[u] + w;
25                   prev[v] = u;
26                   pq.push({dist[v], v});
27               }
28           }
29       }
30   }
31
32   cout << "\nитераций совершено: " << iterations << "\n";
33   return {dist, prev};
34 }

```

Листинг 9: Алгоритм Дейкстры

2.10 Алгоритм Форда-Фалкерсона

Алгоритм поиска максимального потока в сети методом меток. Для каждой вершины хранится метка `LabelInfo { sign, neighbor, delta }` — знак, предшественник в увеличивающем пути и пропускная способность бутылочного горлышка. На каждой итерации сначала помечаются все достижимые вершины (по прямым рёбрам с остаточной пропускной способностью и обратным с ненулевым потоком), затем при достижении стока поток обновляется по пути от стока к истоку, а пометки сбрасываются и поиск повторяется (метка `M`). Алгоритм завершается, когда новых пометок сделать не удалось.

Входные данные: матрица смежности, матрица пропускных способностей, источник s , сток t

Выходные данные: матрица потока F , величина максимального потока

```

1 void fordFulkerson(MyGraph& graph, int s, int t) {
2     int n = graph.verticesCount;
3
4     vector<vector<int>> F(n, vector<int>(n, 0));
5     vector<LabelInfo> P(n);
6     vector<int> S(n);
7     vector<int> N(n);
8     const int INF_VAL = 1e9;
9
10    M:
11    for (int v = 0; v < n; v++) {
12        S[v] = 0;
13        N[v] = 0;
14        P[v] = {'+', -1, 0};
15    }
16    S[s] = 1;
17    P[s] = {'+', -1, INF_VAL};
18
19    int a = 0;
20
21    while (true) {
22        a = 0;
23
24        for (int v = 0; v < n; v++) {
25            if (S[v] == 1 && N[v] == 0) {
26                for (int u = 0; u < n; u++) {
27                    if (graph.adjMatrix[v][u] == 1) {
28                        if (S[u] == 0 && F[v][u] < graph.capacityMatrix[v][u])
29                            ↪ {
30                                S[u] = 1;
31                                int delta = min(P[v].delta,
32                                    ↪ graph.capacityMatrix[v][u] - F[v][u]);
33                                P[u] = {'+', v, delta};
34                                a = 1;
35                            }
36                        }
37                    }
38                }
39            }
40
41            for (int u = 0; u < n; u++) {
42                if (graph.adjMatrix[u][v] == 1) {
43                    if (S[u] == 0 && F[u][v] > 0) {
44                        S[u] = 1;
45                        int delta = min(P[v].delta, F[u][v]);
46                        P[u] = {'-', v, delta};
47                        a = 1;
48                    }
49                }
50            }
51
52            N[v] = 1;
53        }
54    }
55
56    }

```

```

51
52     if (S[t] == 1) {
53         int delta = P[t].delta;
54         int x = t;
55         while (x != s) {
56             int p_node = P[x].neighbor;
57             if (P[x].sign == '+') {
58                 F[p_node][x] += delta;
59             } else {
60                 F[x][p_node] -= delta;
61             }
62             x = p_node;
63         }
64         goto M;
65     }
66
67     if (a == 0) break;
68 }
69
70 int maxFlow = 0;
71 for (int j = 0; j < n; j++) {
72     maxFlow += F[s][j];
73 }
74
75 graph.flowMatrix = F;
76 graph.lastMaxFlow = maxFlow;
77 graph.lastSource = s;
78 graph.lastSink = t;
79 }

```

Листинг 10: Алгоритм Форда-Фалкерсона

2.11 Поток минимальной стоимости

Алгоритм вычисляет поток величины $\lfloor 2/3 \cdot F_{\max} \rfloor$ с минимальной суммарной стоимостью на основе ранее найденного максимального потока. На каждой итерации строится модифицированный граф: прямые рёбра имеют остаточную пропускную способность и положительную стоимость, обратные — пропускную способность, равную текущему потоку, и противоположную стоимость. По модифицированному графу алгоритм Дейкстры находит кратчайший путь от источника к стоку, по которому пропускается поток размером в бутылочное горлышко.

Входные данные: матрица смежности, матрица пропускных способностей, матрица стоимостей, значение максимального потока, источник и сток (из предыдущего запуска Форда-Фалкерсона)

Выходные данные: матрица потока минимальной стоимости, величина достигнутого потока, суммарная стоимость

```

1 void minCostFlowUI(MyGraph& graph, RandomGenerator& rng) {
2     if (graph.hasCapacities == false) {
3         cout << "должна быть создана матрица стоимостей\n";
4         return;
5     }
6     if (graph.lastMaxFlow <= 0) {
7         cout << "сначала выполните алгоритм Форда-Фалкерсона\n";
8         return;
9     }
10
11     int n = graph.verticesCount;
12     int s = graph.lastSource;
13     int t = graph.lastSink;
14     int target = (2 * graph.lastMaxFlow) / 3;
15
16     cout << "\nмаксимальный поток: " << graph.lastMaxFlow << endl;
17     cout << "целевой поток [2/3 * max] = " << target << endl;
18     if (target == 0) return;
19
20     vector<vector<int>> flow(n, vector<int>(n, 0));
21     int currentFlow = 0;
22     int totalCost = 0;
23
24     while (currentFlow < target) {
25         vector<vector<int>> modAdj(n, vector<int>(n, 0));
26         vector<vector<int>> modCap(n, vector<int>(n, 0));
27         vector<vector<int>> modCost(n, vector<int>(n, 0));
28
29         for (int i = 0; i < n; i++) {
30             for (int j = 0; j < n; j++) {
31                 if (graph.adjMatrix[i][j] == 1) {
32                     if (flow[i][j] > 0) {
33                         modAdj[j][i] = 1;
34                         modCap[j][i] = flow[i][j];
35                         modCost[j][i] = -graph.costMatrix[i][j];
36                     }
37                     if (flow[i][j] < graph.capacityMatrix[i][j]) {
38                         modAdj[i][j] = 1;
39                         modCap[i][j] = graph.capacityMatrix[i][j] -
40                             flow[i][j];
41                         modCost[i][j] = graph.costMatrix[i][j];
42                     }
43                 }
44             }
45
46             auto [dist, prev] = dijkstra(n, s, modAdj, modCost);
47             if (dist[t] == INF) break;
48
49             int delta = target - currentFlow;
50             for (int x = t; x != s; x = prev[x])
51                 delta = min(delta, modCap[prev[x]][x]);

```

```

52
53     for (int x = t; x != s; x = prev[x]) {
54         int p = prev[x];
55         if (modCost[p][x] >= 0) {
56             flow[p][x] += delta;
57             totalCost += delta * graph.costMatrix[p][x];
58         } else {
59             flow[x][p] -= delta;
60             totalCost -= delta * graph.costMatrix[x][p];
61         }
62     }
63     currentFlow += delta;
64 }
65
66 cout << "величина потока: " << currentFlow << endl;
67 cout << "суммарная стоимость: " << totalCost << endl;
68 printMatrix(flow, "матрица потока минимальной стоимости");
69 }

```

Листинг 11: Поток минимальной стоимости

2.12 Матричная теорема Кирхгофа

Алгоритм вычисляет число остовных деревьев графа, используя матричную теорему Кирхгофа. Строит матрицу Кирхгофа и вычисляет определитель её минора через метод Гаусса, который по теореме равен числу остовных деревьев.

Входные данные: матрица смежности

Выходные данные: количество остовных деревьев

```

1 double determinant(vector<vector<double>> mat) {
2     int n = mat.size();
3     if (n == 0) return 1.0;
4     double det = 1.0;
5
6     for (int col = 0; col < n; col++) {
7         int pivot = -1;
8         for (int row = col; row < n; row++) {
9             if (fabs(mat[row][col]) > 1e-9) { pivot = row; break; }
10        }
11        if (pivot == -1) return 0.0;
12
13        if (pivot != col) {
14            swap(mat[pivot], mat[col]);
15            det *= -1;
16        }
17
18        det *= mat[col][col];

```

```

19         for (int row = col + 1; row < n; row++) {
20             double f = mat[row][col] / mat[col][col];
21             for (int k = col; k < n; k++)
22                 mat[row][k] -= f * mat[col][k];
23         }
24     }
25     return det;
26 }
27
28 void kirchhoff(MyGraph& graph) {
29     if (graph.isDirected) {
30         cout << "теорема Кирхгофа применяется только к неориентированным
31             ↪ графам\n";
32         return;
33     }
34     int n = graph.verticesCount;
35     if (n <= 1) {
36         cout << "\nчисло остовных деревьев: 1\n";
37         return;
38     }
39
40     vector<vector<int>> B(n, vector<int>(n, 0));
41     for (int i = 0; i < n; i++)
42         for (int j = 0; j < n; j++)
43             if (i != j && graph.adjMatrix[i][j] == 1) {
44                 B[i][j] = -1;
45                 B[i][i]++;
46             }
47
48     printMatrix(B, "матрица Кирхгофа B(G)");
49
50     vector<vector<double>> minor(n - 1, vector<double>(n - 1));
51     for (int i = 1; i < n; i++)
52         for (int j = 1; j < n; j++)
53             minor[i - 1][j - 1] = B[i][j];
54
55     long long result = (long long)round(determinant(minor));
56     cout << "\nчисло остовных деревьев: " << result << endl;
57 }

```

Листинг 12: Матричная теорема Кирхгофа

2.13 Кодирование Прюфера

Алгоритм кодирует остовное дерево в расширенный код Прюфера длины $n - 1$, где каждая запись — пара (сосед, вес ребра). На каждом шаге находится лист с минимальным номером, его сосед и вес ребра записываются в код, а лист удаляется из дерева. Расширенный код позволяет восстановить веса всех рёбер без дополнительного хранения.

Входные данные: матрица смежности остова, матрица весов остова

Выходные данные: код Прюфера — последовательность из $n - 1$ пар (вершина, вес ребра)

```
1 void pruferEncode(MyGraph& graph, const vector<vector<int>>& adjMatrix,
2                     const vector<vector<int>>& weightMatrix) {
3     int n = adjMatrix.size();
4     graph.pruferCode.clear();
5
6     vector<vector<int>> adj = adjMatrix;
7     vector<int> degree(n, 0);
8
9     for (int i = 0; i < n; i++)
10         for (int j = 0; j < n; j++)
11             if (adj[i][j] == 1) degree[i]++;
12
13     // расширенный код: срезаем листья n-1 раз, кодируя все рёбра дерева
14     for (int i = 0; i < n - 1; i++) {
15         int leaf = -1;
16         for (int v = 0; v < n; v++) {
17             if (degree[v] == 1) { leaf = v; break; }
18         }
19
20         if (leaf != -1) {
21             int neighbor = -1;
22             for (int v = 0; v < n; v++) {
23                 if (adj[leaf][v] == 1) { neighbor = v; break; }
24             }
25
26             // кодируем пару: сосед и вес удаляемого ребра (leaf, neighbor)
27             graph.pruferCode.push_back({neighbor,
28                                         ↪ weightMatrix[leaf][neighbor]});
29
30             for (int v = 0; v < n; v++) {
31                 if (adj[leaf][v] == 1) {
32                     adj[leaf][v] = 0;
33                     adj[v][leaf] = 0;
34                 }
35             }
36             degree[leaf] = 0;
37             degree[neighbor]--;
38         }
39     }
```

Листинг 13: Кодирование Прюфера

2.14 Декодирование Прюфера

Алгоритм восстанавливает остовное дерево из расширенного кода Прюфера. Веса всех рёбер восстанавливаются без обращения к исходной матрице весов, поскольку в поле `prufercode` структуры данных графа хранится вектор пар, куда закодированы как номера вершин, так и веса.

Входные данные: расширенный код Прюфера $\{(v_i, w_i)\}_{i=1}^{n-1}$

Выходные данные: восстановленная матрица смежности и матрица весов остова

```
1 void pruferDecode(MyGraph& graph) {
2     int n = graph.verticesCount;
3     const vector<pair<int, int>>& code = graph.pruferCode;
4
5     vector<vector<int>> decodedAdj(n, vector<int>(n, 0));
6     vector<vector<int>> decodedWeight(n, vector<int>(n, 0));
7
8     // степени: count(v в коде) + 1, затем у последней оставшейся вершины -1
9     vector<int> degree(n, 1);
10    for (const auto& p : code) degree[p.first]++;
11    if (!code.empty()) {
12        int r = code.back().first;
13        degree[r]--;
14    }
15
16    // расширенный код длины n-1 кодирует все рёбра - отдельного блока не
17    ↪ нужно
18    for (int i = 0; i < (int)code.size(); i++) {
19        int leaf = -1;
20        for (int v = 0; v < n; v++) {
21            if (degree[v] == 1) { leaf = v; break; }
22        }
23
24        int neighbor = code[i].first;
25        decodedAdj[leaf][neighbor] = 1;
26        decodedAdj[neighbor][leaf] = 1;
27
28        // вес берём из самого кода Прюфера
29        int weight = code[i].second;
30        decodedWeight[leaf][neighbor] = weight;
31        decodedWeight[neighbor][leaf] = weight;
32
33        degree[leaf]--;
34        degree[neighbor]--;
35    }
36
37    graph.ostAdjMatrix = decodedAdj;
38    graph.ostWeightMatrix = decodedWeight;
39 }
```

2.15 Алгоритм Краскала

Реализация алгоритма Краскала согласно псевдокоду из лекций. Находит минимальное остовное дерево, используя жадный выбор ребер в порядке возрастания весов с проверкой циклов.

Входные данные: матрица смежности, матрица весов

Выходные данные: матрица смежности и матрица весов минимального остова

```

1 struct Edge {
2     int u, v, weight;
3     bool operator<(const Edge& other) const {
4         return weight < other.weight;
5     }
6 };
7
8 int findParent(int x, vector<int>& parent) {
9     if (parent[x] != x) parent[x] = findParent(parent[x], parent);
10    return parent[x];
11 }
12
13 void kruskal(MyGraph& graph) {
14     int n = graph.verticesCount;
15
16     vector<Edge> edges;
17     for (int i = 0; i < n; i++) {
18         for (int j = i + 1; j < n; j++) {
19             if (graph.adjMatrix[i][j] == 1) {
20                 edges.push_back({i, j, graph.weightMatrix[i][j]});
21             }
22         }
23     }
24
25     sort(edges.begin(), edges.end());
26
27     vector<int> parent(n);
28     for (int i = 0; i < n; i++) parent[i] = i;
29
30     vector<Edge> mst;
31     int k = 0;
32     int edgeCount = edges.size();
33
34     for (int i = 0; i < n - 1; i++) {
35         while (k < edgeCount) {
36             int pu = findParent(edges[k].u, parent);
37             int pv = findParent(edges[k].v, parent);
38             if (pu == pv) k++;
39             else break;

```

```

40     }
41
42     if (k < edgeCount) {
43         mst.push_back(edges[k]);
44         int pu = findParent(edges[k].u, parent);
45         int pv = findParent(edges[k].v, parent);
46         parent[pu] = pv;
47         k++;
48     }
49 }
50
51 graph.ostAdjMatrix.assign(n, vector<int>(n, 0));
52 graph.ostWeightMatrix.assign(n, vector<int>(n, 0));
53
54 int totalWeight = 0;
55 for (const auto& edge : mst) {
56     graph.ostAdjMatrix[edge.u][edge.v] = 1;
57     graph.ostAdjMatrix[edge.v][edge.u] = 1;
58     graph.ostWeightMatrix[edge.u][edge.v] = edge.weight;
59     graph.ostWeightMatrix[edge.v][edge.u] = edge.weight;
60     totalWeight += edge.weight;
61 }
62
63 graph.lastMst = totalWeight;
64 }

```

Листинг 15: Алгоритм Краскала (чистый алгоритм)

2.16 Максимальное независимое множество вершин

Реализация алгоритма построения максимальных независимых множеств с возвратом (backtracking). Использует три множества $S[k]$ (текущее независимое множество), $Q^+[k]$ (кандидаты для расширения) и $Q^-[k]$ (уже использованные вершины), что обеспечивает обход всех максимальных независимых множеств без дубликатов. Среди найденных выбирается множество максимальной мощности.

Входные данные: матрица смежности (исходный граф или остов)

Выходные данные: вершины максимального независимого множества и его размер

```
1 void maxIndependentSet(MyGraph& graph) {
2     if (!graph.isGenerated) {
3         cout << "сначала сгенерируйте граф\n";
4         return;
5     }
6
7     int n = graph.verticesCount;
8     if (n == 0) {
9         cout << "пустой граф\n";
10        return;
11    }
12
13    int sourceChoice;
14    cout << "на каком графе искать максимальное независимое множество?\n";
15    cout << "1 - на исходном графе\n";
16    cout << "2 - на остове\n";
17    cout << "выбор: ";
18    cin >> sourceChoice;
19
20    vector<vector<int>> baseAdj;
21    string sourceName;
22    if (sourceChoice == 1) {
23        baseAdj = graph.adjMatrix;
24        sourceName = "исходный граф";
25    } else if (sourceChoice == 2) {
26        bool hasOst = false;
27        for (int i = 0; i < n && !hasOst; i++)
28            for (int j = 0; j < n && !hasOst; j++)
29                if (graph.ostAdjMatrix[i][j] == 1) hasOst = true;
30        if (!hasOst) {
31            cout << "сначала постройте остов (пункт 12)\n";
32            return;
33        }
34        baseAdj = graph.ostAdjMatrix;
35        sourceName = "остов";
36    } else {
37        cout << "неверный выбор\n";
38        return;
39    }
```

```

39     }
40     vector<set<int>> gamma(n); // список соседей вершины i
41     for (int i = 0; i < n; i++) {
42         for (int j = 0; j < n; j++) {
43             if (i != j && (baseAdj[i][j] == 1 || baseAdj[j][i] == 1)) {
44                 gamma[i].insert(j);
45             }
46         }
47     }
48
49     vector<vector<int>> S(n + 2); // уже добавленные вершины в множество
50     vector<set<int>> Qplus(n + 2); // кандидаты
51     vector<set<int>> Qminus(n + 2); // запрещенные вершины
52
53     int k = 0;
54     for (int v = 0; v < n; v++) Qplus[0].insert(v);
55
56     vector<int> best;
57
58 M1:
59     {
60         // шаг вперед: select v Q+[k]
61         int v = *Qplus[k].begin();
62         S[k + 1] = S[k];
63         S[k + 1].push_back(v);
64         Qminus[k + 1] = Qminus[k];
65         for (int u : gamma[v]) Qminus[k + 1].erase(u);
66         Qplus[k + 1] = Qplus[k];
67         Qplus[k + 1].erase(v);
68         for (int u : gamma[v]) Qplus[k + 1].erase(u);
69         k++;
70     }
71
72 M2:
73     {
74         bool earlyReturn = false;
75         for (int u : Qminus[k]) {
76             bool hasIntersection = false;
77             for (int w : gamma[u]) {
78                 if (Qplus[k].count(w)) {
79                     hasIntersection = true;
80                     break;
81                 }
82             }
83             if (!hasIntersection) {
84                 earlyReturn = true;
85                 break;
86             }
87         }
88         if (earlyReturn) goto M3;
89
90         if (Qplus[k].empty()) {
91             if (Qminus[k].empty()) {

```

```

92         if ((int)S[k].size() > (int)best.size()) {
93             best = S[k];
94         }
95     }
96     goto M3;
97 } else {
98     goto M1;
99 }
100 }
101
102 M3:
103 {
104     if (k == 0) goto END_LOOP;
105     int v = S[k].back();
106     k--;
107     S[k] = S[k + 1];
108     S[k].pop_back();
109     Qminus[k].insert(v);
110     Qplus[k].erase(v);
111     if (k == 0 && Qplus[k].empty()) goto END_LOOP;
112     else goto M2;
113 }
114
115 END_LOOP:
116 cout << "\налгоритм завершён (" << sourceName << ")\n";
117 if (best.empty()) {
118     cout << "максимальное независимое множество не найдено\n";
119     return;
120 }
121 cout << "размер: " << best.size() << "\n";
122 cout << "вершины: { ";
123 for (int v : best) cout << v << " ";
124 cout << "}\n";
125 }

```

Листинг 16: Максимальное независимое множество вершин

2.17 Эйлеров цикл (модификация и алгоритм Флёри)

Реализация проверки графа на эйлеровость, модификации при необходимости и построения эйлерова цикла. Граф эйлеров, если все вершины имеют чётную степень (связность гарантирована построением). Если есть вершины нечётной степени, они обрабатываются парами: добавляется ребро между ними (приоритет добавления), либо через посредника, либо удаляется существующее. Эйлеров цикл строится алгоритмом Флёри строго по псевдокоду из лекций: используется стек S и списки смежности $\Gamma[v]$; при тупике вершина вытаскивается в результирующий цикл.

Входные данные: неориентированный граф (матрица смежности)

Выходные данные: лог модификаций и последовательность вершин эйлерова цикла

```
1 void eulerCycle(MyGraph& graph) {
2     if (!graph.isGenerated) {
3         cout << "сначала сгенерируйте граф\n";
4         return;
5     }
6     if (graph.isDirected) {
7         cout << "эйлеров цикл строится только для неориентированного графа\n";
8         return;
9     }
10
11     int n = graph.verticesCount;
12     if (n <= 1) {
13         cout << "граф слишком мал - эйлеров цикл тривиален (пуст)\n";
14         return;
15     }
16
17     // работаем на копии, исходный граф не изменяем
18     vector<vector<int>> adj = graph.adjMatrix;
19
20     auto degreeOf = [&](int v) {
21         int d = 0;
22         for (int u = 0; u < n; u++) d += adj[v][u];
23         return d;
24     };
25
26     vector<int> odd;
27     for (int v = 0; v < n; v++) {
28         if (degreeOf(v) % 2 != 0) odd.push_back(v);
29     }
30
31     if (odd.empty()) {
32         cout << "все степени чётные - граф ЭЙЛЕРОВ, модификация не
           ↳ требуется\n";
33     } else {
34         cout << "вершин с нечётной степенью: " << odd.size() << " -> ";
35         for (int v : odd) cout << v << " ";
```

```

36     cout << "\nграф НЕ эйлеров, выполняется модификация:\n";
37
38     for (size_t i = 0; i + 1 < odd.size(); i += 2) {
39         int a = odd[i], b = odd[i + 1];
40         if (adj[a][b] == 0) {
41             adj[a][b] = adj[b][a] = 1;
42             cout << "    + добавлено ребро (" << a << ", " << b << ")\n";
43         } else {
44             int w = -1;
45             for (int t = 0; t < n; t++) {
46                 if (t != a && t != b && adj[a][t] == 0 && adj[b][t] == 0)
47                     {
48                         w = t;
49                         break;
50                     }
51             }
52             if (w != -1) {
53                 adj[a][w] = adj[w][a] = 1;
54                 adj[w][b] = adj[b][w] = 1;
55                 cout << "    + ребро (" << a << ", " << b << ") уже
56                     ↳ существует; добавлены рёбра ("
57                     << a << ", " << w << ") и (" << w << ", " << b << ")
58                     ↳ \n";
59             } else {
60                 adj[a][b] = adj[b][a] = 0;
61                 cout << "    - посредник не найден; удалено ребро (" << a <<
62                     ↳ ", " << b << ")\n";
63             }
64         }
65     }
66
67     vector<int> oddAfter;
68     for (int v = 0; v < n; v++)
69         if (degreeOf(v) % 2 != 0) oddAfter.push_back(v);
70
71     if (!oddAfter.empty()) {
72         cout << "ОШИБКА: после модификации остались нечётные вершины\n";
73         return;
74     }
75     cout << "модификация завершена - все степени чётные\n";
76     printMatrix(adj, "матрица смежности после модификации");
77 }
78
79 int edgeCount = 0;
80 for (int i = 0; i < n; i++)
81     for (int j = i + 1; j < n; j++)
82         edgeCount += adj[i][j];
83
84 if (edgeCount == 0) {
85     cout << "\nв графе нет рёбер - эйлеров цикл пуст\n";
86     return;
87 }

```

```

85     int start = 0;
86     for (int v = 0; v < n; v++) {
87         if (degreeOf(v) > 0) { start = v; break; }
88     }
89
90     // алгоритм Флёрри: списки смежности [v] и стек S
91     vector<set<int>> Gamma(n);
92     for (int i = 0; i < n; i++)
93         for (int j = 0; j < n; j++)
94             if (adj[i][j] == 1) Gamma[i].insert(j);
95
96     vector<int> cycle;
97     stack<int> S;
98     S.push(start);
99
100    while (!S.empty()) {
101        int v = S.top();
102        if (Gamma[v].empty()) {
103            S.pop();
104            cycle.push_back(v);
105        } else {
106            int u = *Gamma[v].begin();
107            S.push(u);
108            Gamma[v].erase(u);
109            Gamma[u].erase(v);
110        }
111    }
112
113    cout << "\n=== Эйлеров цикл (алгоритм Флёрри) ===\n";
114    cout << "длина (рёбер): " << cycle.size() - 1 << "\n";
115    cout << "цикл: ";
116    for (size_t i = 0; i < cycle.size(); i++) {
117        cout << cycle[i];
118        if (i + 1 < cycle.size()) cout << " -> ";
119    }
120    cout << "\n";
121 }

```

Листинг 17: Эйлеров цикл (модификация и алгоритм Флёрри)

2.18 Фундаментальная система разрезов

Реализация построения фундаментальной системы разрезов на основе минимального остова. Для каждого древесного ребра $e = (a, b)$ выполняется BFS по остову без этого ребра, что даёт разбиение вершин на множества V_1 и V_2 . Фундаментальный разрез $S_e = E(V_1, V_2)$ — все рёбра исходного графа, концы которых лежат в разных частях разбиения. Количество фундаментальных разрезов равно коцикломатическому числу $m^*(G) = p - 1$. Любой разрез графа может быть получен симметрической разностью некоторых фундаментальных разрезов — пользователь вводит номера разрезов для комбинирования.

Входные данные: матрица смежности графа, матрица смежности предварительно построенного остова, номера разрезов для комбинирования

Выходные данные: список фундаментальных разрезов и результат симметрической разности

```
1 static pair<int, int> normEdge(int u, int v) {
2     return {min(u, v), max(u, v)};
3 }
4
5 static void printCut(const set<pair<int, int>>& cut) {
6     cout << "{ ";
7     for (const auto& e : cut) {
8         cout << "(" << e.first << "-" << e.second << ") ";
9     }
10    cout << "}";
11 }
12
13 void fundamentalCuts(MyGraph& graph) {
14     if (!graph.isGenerated) {
15         cout << "сначала сгенерируйте граф\n";
16         return;
17     }
18     if (graph.isDirected) {
19         cout << "разрезы строятся только для неориентированного графа\n";
20         return;
21     }
22
23     int n = graph.verticesCount;
24     if (n <= 1) {
25         cout << "граф слишком мал - разрезов нет\n";
26         return;
27     }
28
29     bool hasOst = false;
30     for (int i = 0; i < n && !hasOst; i++)
31         for (int j = 0; j < n && !hasOst; j++)
32             if (graph.ostAdjMatrix[i][j] == 1) hasOst = true;
33     if (!hasOst) {
```

```

34     cout << "сначала постройте остов (пункт 12)\n";
35     return;
36 }
37
38 // собираем рёбра остова, их p-1
39 vector<pair<int, int>> treeEdges;
40 for (int i = 0; i < n; i++)
41     for (int j = i + 1; j < n; j++)
42         if (graph.ostAdjMatrix[i][j] == 1)
43             treeEdges.push_back({i, j});
44
45 int m = treeEdges.size(); // коцикломатическое число = p-1
46 vector<set<pair<int, int>>> cuts(m);
47
48 for (int idx = 0; idx < m; idx++) {
49     int a = treeEdges[idx].first;
50     int b = treeEdges[idx].second;
51
52     // BFS по остову БЕЗ ребра (a,b): находим V1, содержащую a
53     vector<bool> inV1(n, false);
54     queue<int> q;
55     q.push(a);
56     inV1[a] = true;
57
58     while (!q.empty()) {
59         int u = q.front();
60         q.pop();
61         for (int w = 0; w < n; w++) {
62             if (graph.ostAdjMatrix[u][w] == 1 && !inV1[w]) {
63                 if ((u == a && w == b) || (u == b && w == a)) continue;
64                 inV1[w] = true;
65                 q.push(w);
66             }
67         }
68     }
69
70     // S_e = все рёбра графа G между V1 и V2
71     for (int u = 0; u < n; u++)
72         for (int w = u + 1; w < n; w++)
73             if (graph.adjMatrix[u][w] == 1 && (inV1[u] != inV1[w]))
74                 cuts[idx].insert(normEdge(u, w));
75 }
76
77 cout << "\n=== Фундаментальная система разрезов ===\n";
78 cout << "коцикломатическое число m*(G) = p - 1 = " << m << "\n";
79 for (int idx = 0; idx < m; idx++) {
80     cout << "S" << (idx + 1) << " (удаленное ребро ("
81         << treeEdges[idx].first << "-" << treeEdges[idx].second << ")):
82         << "\n";
83     printCut(cuts[idx]);
84     cout << "\n";
85 }

```

```

86     cout << "\nполучение разреза через симметрическую разность
      ↪ фундаментальных.\n";
87     cout << "сколько фундаментальных разрезов скомбинировать? (0 -
      ↪ пропустить): ";
88     int cnt;
89     cin >> cnt;
90
91     if (cnt <= 0) return;
92
93     set<pair<int, int>> result;
94     bool ok = true;
95     for (int i = 0; i < cnt; i++) {
96         cout << "введите номер разреза (1.." << m << "): ";
97         int num;
98         cin >> num;
99         if (num < 1 || num > m) {
100             cout << "неверный номер\n";
101             ok = false;
102             break;
103         }
104
105         for (const auto& e : cuts[num - 1]) {
106             if (result.count(e)) result.erase(e);
107             else result.insert(e);
108         }
109     }
110
111     if (!ok) return;
112
113     cout << "\nрезультат (симметрическая разность выбранных разрезов): ";
114     printCut(result);
115     cout << "\n";
116     if (result.empty())
117         cout << "(пустое множество - выбранные разрезы взаимно
      ↪ сократились)\n";
118 }

```

Листинг 18: Фундаментальная система разрезов

3 Результаты работы

Программа предоставляет пользователю консольное меню, позволяющее выполнить все необходимые операции над графами. Ниже приведены результаты работы программы для каждого пункта меню.

На рисунке 3 представлено главное меню программы.

```
меню:
===== Lab 1 =====
1. сгенерировать граф
2. сгенерировать весовую матрицу
3. сгенерировать матрицы стоимостей и пропускных способностей
4. найти центр и диаметр
5. метод шимбелла
6. подсчет маршрутов
===== Lab 2 =====
7. обход графа в глубину (dfs)
8. поиск кратчайшего пути (Дейкстра)
===== Lab 3 =====
9. алгоритм Форда-Фалкерсона
10. поток минимальной стоимости [2/3 * max]
===== Lab 4 =====
11. найти число остовных деревьев по т. Кирхгофа
12. минимальный остов (алгоритм Краскала)
13. максимальное независимое множество вершин
===== Lab 5 =====
14. проверка на эйлеров граф
15. фундаментальная система разрезов
0. выход
выбор: █
```

Рис. 3: Меню программы

3.1 Лабораторная работа №1

Первая лабораторная работа включает шесть пунктов меню: генерация графа, генерация весовой матрицы, генерация матриц стоимостей и пропускных способностей, поиск центра и диаметра графа, метод Шимбелла и подсчёт маршрутов.

Далее на рисунке 4 можно увидеть пример генерации графа.

```

выбор: 1
введите количество вершин: 10
тип графа (0 – неориентированный, 1 – ориентированный): 1

Был сгенерирован связный граф

граф успешно сгенерирован за 1 попыток
заданные степени вершин (по распределению):
вершина 0: 4
вершина 1: 2
вершина 2: 6
вершина 3: 8
вершина 4: 1
вершина 5: 2
вершина 6: 5
вершина 7: 2
вершина 8: 7
вершина 9: 5

реальные степени вершин (по построенной матрице):
вершина 0: 4
вершина 1: 2
вершина 2: 6
вершина 3: 6
вершина 4: 1
вершина 5: 2
вершина 6: 3
вершина 7: 2
вершина 8: 1
вершина 9: 0

количество ребер: 27

матрица смежности:

```

	0	1	2	3	4	5	6	7	8	9
0	0	1	1	1	0	0	1	0	0	0
1	0	0	0	0	0	1	1	0	0	0
2	0	0	0	1	1	1	1	0	1	1
3	0	0	0	0	1	1	1	1	1	1
4	0	0	0	0	0	0	0	0	0	1
5	0	0	0	0	0	0	1	0	1	0
6	0	0	0	0	0	0	0	1	1	1
7	0	0	0	0	0	0	0	0	1	1
8	0	0	0	0	0	0	0	0	0	1
9	0	0	0	0	0	0	0	0	0	0

Рис. 4: Пункт 1. Пример генерации графа

Далее на рисунке 5 приведён пример генерации весовой матрицы.

```

выбор: 2
выберите тип весов:
1 – только положительные
2 – только отрицательные
3 – смешанные
выбор: 3

генерация весовой матрицы:

матрица весов:
      0  1  2  3  4  5  6  7  8  9
-----
0|  0 -3 -2  5  0  0  2  0  0  0
1|  0  0  0  0  0 -2  3  0  0  0
2|  0  0  0 -2  2 -8  2  0 -3  4
3|  0  0  0  0 -4  2  3  2 -2 -2
4|  0  0  0  0  0  0  0  0  0  2
5|  0  0  0  0  0  0  3  0 -4  0
6|  0  0  0  0  0  0  0  1  2  3
7|  0  0  0  0  0  0  0  0 -2 -7
8|  0  0  0  0  0  0  0  0  0  5
9|  0  0  0  0  0  0  0  0  0  0

```

Рис. 5: Пункт 2. Пример генерации весовой матрицы

Далее на рисунке 6 можно увидеть генерацию матриц стоимостей и пропускных способностей.

выбор: 3

матрица пропускных способностей:

	0	1	2	3	4	5	6	7	8	9
0	0	12	4	6	0	0	8	0	0	0
1	0	0	0	0	0	11	3	0	0	0
2	0	0	0	2	2	3	3	0	4	2
3	0	0	0	0	3	6	3	4	6	2
4	0	0	0	0	0	0	0	0	0	2
5	0	0	0	0	0	0	3	0	6	0
6	0	0	0	0	0	0	0	4	4	6
7	0	0	0	0	0	0	0	0	2	4
8	0	0	0	0	0	0	0	0	0	1
9	0	0	0	0	0	0	0	0	0	0

матрица стоимостей:

	0	1	2	3	4	5	6	7	8	9
0	0	1	6	17	0	0	10	0	0	0
1	0	0	0	0	0	5	1	0	0	0
2	0	0	0	5	3	3	4	0	3	5
3	0	0	0	0	2	9	7	3	1	5
4	0	0	0	0	0	0	0	0	0	3
5	0	0	0	0	0	0	3	0	3	0
6	0	0	0	0	0	0	0	4	4	2
7	0	0	0	0	0	0	0	0	1	2
8	0	0	0	0	0	0	0	0	0	5
9	0	0	0	0	0	0	0	0	0	0

Рис. 6: Пункт 3. Пример генерации матриц стоимостей и пропускных способностей

Далее на рисунке 7 приведён результат поиска центра и диаметра графа.

```
выбор: 4
эксцентриситет вершины 0: 2
эксцентриситет вершины 1: 2
эксцентриситет вершины 2: 2
эксцентриситет вершины 3: 1
эксцентриситет вершины 4: 1
эксцентриситет вершины 5: 2
эксцентриситет вершины 6: 1
эксцентриситет вершины 7: 1
эксцентриситет вершины 8: 1
эксцентриситет вершины 9: 0

радиус графа: 0
диаметр графа: 2
центр графа (вершины с эксцентриситетом = радиусу): 9
диаметральные вершины (эксцентриситет = диаметру): 0 1 2 5
```

Рис. 7: Пункт 4. Пример поиска центра и диаметра графа

Далее на рисунке 8 показан пример работы метода Шимбелла.

```

выбор: 5
введите длину пути (количество ребер): 3

матрица минимальных путей:
      0  1  2  3  4  5  6  7  8  9
-----
0 |  -  -  -  -  -8 -2 -7 -2 -14 -6
1 |  -  -  -  -  -  -  -  2  2  -3
2 |  -  -  -  -  -  -  3 -4 -4 -7
3 |  -  -  -  -  -  -  -  6  2 -3
4 |  -  -  -  -  -  -  -  -  -  -
5 |  -  -  -  -  -  -  -  -  2 -3
6 |  -  -  -  -  -  -  -  -  -  4
7 |  -  -  -  -  -  -  -  -  -  -
8 |  -  -  -  -  -  -  -  -  -  -
9 |  -  -  -  -  -  -  -  -  -  -

матрица максимальных путей:
      0  1  2  3  4  5  6  7  8  9
-----
0 |  -  -  -  -  -8 -2 10  9 10 11
1 |  -  -  -  -  -  -  -  2  3 10
2 |  -  -  -  -  -  -  3  2  3  9
3 |  -  -  -  -  -  -  -  6  7 10
4 |  -  -  -  -  -  -  -  -  -  -
5 |  -  -  -  -  -  -  -  -  2 10
6 |  -  -  -  -  -  -  -  -  -  4
7 |  -  -  -  -  -  -  -  -  -  -
8 |  -  -  -  -  -  -  -  -  -  -
9 |  -  -  -  -  -  -  -  -  -  -

```

Рис. 8: Пункт 5. Пример работы метода Шимбелла

Далее на рисунке 9 приведён пример подсчёта маршрутов в графе.

```

выбор: 6
введите начальную вершину: 2
введите конечную вершину: 5

матрица сумм маршрутов (длины от 0 до 9):
  0  1  2  3  4  5  6  7  8  9
-----
0|  1  1  1  2  3  4  9 11 27 53
1|  -  1  -  -  -  1  2  2  5  9
2|  -  -  1  1  2  2  4  5 13 26
3|  -  -  -  1  1  1  2  3  7 14
4|  -  -  -  -  1  -  -  -  -  1
5|  -  -  -  -  -  1  1  1  3  5
6|  -  -  -  -  -  -  1  1  2  4
7|  -  -  -  -  -  -  -  1  1  2
8|  -  -  -  -  -  -  -  -  1  1
9|  -  -  -  -  -  -  -  -  -  1

общее количество маршрутов из вершины 2 в вершину 5 (длины от 0 до 9): 2

```

Рис. 9: Пункт 6. Пример подсчёта маршрутов

3.2 Лабораторная работа №2

Вторая лабораторная работа включает два пункта меню: обход графа в глубину и поиск кратчайшего пути алгоритмом Дейкстры.

Далее на рисунке 10 можно увидеть пример обхода графа в глубину.

```

выбор: 7
введите начальную вершину: 1

обход графа в глубину (dfs):
последовательность вершин: 1 2 3 4 8 9 5 6 7
итераций совершено: 90

```

Рис. 10: Пункт 7. Пример обхода графа в глубину (DFS)

Далее на рисунке 11 приведён пример поиска кратчайшего пути алгоритмом Дейкстры.

```
выбор: 8
введите начальную вершину: 2
введите конечную вершину: 5

итераций совершено: 190

вектор расстояний от вершины 2:
вершина 0: INF
вершина 1: INF
вершина 2: 0
вершина 3: 3
вершина 4: 5
вершина 5: 1
вершина 6: -6
вершина 7: -7
вершина 8: -11
вершина 9: -15

кратчайший путь из вершины 2 в вершину 5:
расстояние: 1
путь: 2 -> 3 -> 4 -> 5
```

Рис. 11: Пункт 8. Пример поиска кратчайшего пути (алгоритм Дейкстры)

3.3 Лабораторная работа №3

Третья лабораторная работа включает два пункта меню: алгоритм Форда–Фалкерсона и поиск потока минимальной стоимости величиной $2/3$ от максимального.

Далее на рисунке 12 показан пример работы алгоритма Форда–Фалкерсона.

```

выбор: 9
доступные истоки (нет входящих рёбер): 0 1 2
введите источник: 1
автоматически выбран сток: 9

максимальный поток: 2

матрица потока F:
      0  1  2  3  4  5  6  7  8  9
-----
0|    0  0  0  0  0  0  0  0  0  0
1|    0  0  0  0  0  2  0  0  0  0
2|    0  0  0  0  0  0  0  0  0  0
3|    0  0  0  0  0  0  0  0  0  0
4|    0  0  0  0  0  0  0  0  0  0
5|    0  0  0  0  0  0  0  0  0  2
6|    0  0  0  0  0  0  0  0  0  0
7|    0  0  0  0  0  0  0  0  0  0
8|    0  0  0  0  0  0  0  0  0  0
9|    0  0  0  0  0  0  0  0  0  0

```

Рис. 12: Пункт 9. Пример работы алгоритма Форда–Фалкерсона

Далее на рисунке 13 приведён пример поиска потока минимальной стоимости.

```

выбор: 10

максимальный поток (из предыдущего запуска): 2
целевой поток  $[2/3 * \max] = 1$ 

итераций совершено: 60

поток минимальной стоимости найден
величина потока: 1
суммарная стоимость: 4

матрица потока минимальной стоимости:

```

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	1
6	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0

Рис. 13: Пункт 10. Пример поиска потока минимальной стоимости $[2/3 \cdot \max]$

3.4 Лабораторная работа №4

Четвёртая лабораторная работа включает три пункта меню: поиск числа остовных деревьев по теореме Кирхгофа, построение минимального остова алгоритмом Краскала и поиск максимального независимого множества вершин.

Далее на рисунке 14 можно увидеть пример подсчёта числа остовных деревьев по теореме Кирхгофа.

выбор: 11

матрица Кирхгофа $B(G)$:

	0	1	2	3	4	5	6	7	8	9
0	4	-1	0	0	0	-1	-1	0	0	-1
1	-1	4	-1	0	-1	0	0	-1	0	0
2	0	-1	7	-1	-1	0	-1	-1	-1	-1
3	0	0	-1	3	0	-1	0	0	-1	0
4	0	-1	-1	0	3	0	0	0	0	-1
5	-1	0	0	-1	0	4	-1	-1	0	0
6	-1	0	-1	0	0	-1	5	-1	0	-1
7	0	-1	-1	0	0	-1	-1	6	-1	-1
8	0	0	-1	-1	0	0	0	-1	4	-1
9	-1	0	-1	0	-1	0	-1	-1	-1	6

число остовных деревьев: 103020

Рис. 14: Пункт 11. Пример подсчёта числа остовных деревьев по теореме Кирхгофа

Далее на рисунке 15 приведён пример построения минимального остова алгоритмом Краскала.

```

выбор: 12

общий вес остова: -48

минимальный остова (алгоритм Краскала):

матрица смежности остова:
  0  1  2  3  4  5  6  7  8  9
0| 0  1  0  0  0  0  0  0  0  0
1| 1  0  1  0  1  0  0  1  0  0
2| 0  1  0  1  0  0  0  0  1  0
3| 0  0  1  0  0  1  0  0  0  0
4| 0  1  0  0  0  0  0  0  0  0
5| 0  0  0  1  0  0  0  0  0  0
6| 0  0  0  0  0  0  0  1  0  1
7| 0  1  0  0  0  0  1  0  0  0
8| 0  0  1  0  0  0  0  0  0  0
9| 0  0  0  0  0  0  1  0  0  0

матрица весов остова:
  0  1  2  3  4  5  6  7  8  9
0| 0 -9  0  0  0  0  0  0  0  0
1| -9  0  1  0 -10  0  0 -3  0  0
2|  0  1  0  1  0  0  0  0 -3  0
3|  0  0  1  0  0 -14  0  0  0  0
4|  0 -10  0  0  0  0  0  0  0  0
5|  0  0  0 -14  0  0  0  0  0  0
6|  0  0  0  0  0  0  0 -5  0 -6
7|  0 -3  0  0  0  0 -5  0  0  0
8|  0  0 -3  0  0  0  0  0  0  0
9|  0  0  0  0  0  0 -6  0  0  0

код Прюфера (вершина, вес), длина n-1: (1, -9) (1, -10) (3, -14) (2, 1) (2, -3) (1, 1) (7, -3) (6, -5) (9, -6)

восстановленный остова (декодирование кода Прюфера):

матрица смежности восстановленного остова:
  0  1  2  3  4  5  6  7  8  9
0| 0  1  0  0  0  0  0  0  0  0
1| 1  0  1  0  1  0  0  1  0  0
2| 0  1  0  1  0  0  0  0  1  0
3| 0  0  1  0  0  1  0  0  0  0
4| 0  1  0  0  0  0  0  0  0  0
5| 0  0  0  1  0  0  0  0  0  0
6| 0  0  0  0  0  0  0  1  0  1
7| 0  1  0  0  0  0  1  0  0  0
8| 0  0  1  0  0  0  0  0  0  0
9| 0  0  0  0  0  0  1  0  0  0

матрица весов восстановленного остова:
  0  1  2  3  4  5  6  7  8  9
0| 0 -9  0  0  0  0  0  0  0  0
1| -9  0  1  0 -10  0  0 -3  0  0
2|  0  1  0  1  0  0  0  0 -3  0
3|  0  0  1  0  0 -14  0  0  0  0
4|  0 -10  0  0  0  0  0  0  0  0
5|  0  0  0 -14  0  0  0  0  0  0
6|  0  0  0  0  0  0  0 -5  0 -6
7|  0 -3  0  0  0  0 -5  0  0  0
8|  0  0 -3  0  0  0  0  0  0  0
9|  0  0  0  0  0  0 -6  0  0  0

```

Рис. 15: Пункт 12. Пример построения минимального остова (алгоритм Краскала)

Далее на рисунке 16 показан пример поиска максимального независимого множества вершин.

```

выбор: 13
на каком графе искать максимальное независимое множество?
1 – на исходном графе
2 – на остоле
выбор: 2

алгоритм завершён (остов)
размер: 6
вершины: { 0 2 4 5 7 9 }

```

Рис. 16: Пункт 13. Пример поиска максимального независимого множества вершин

3.5 Лабораторная работа №5

Пятая лабораторная работа включает два пункта меню: проверка графа на эйлеровость и построение фундаментальной системы разрезов.

Далее на рисунке 17 приведён пример проверки графа на эйлеровость и построения эйлерова цикла.

```

выбор: 14
вершин с нечётной степенью: 4 -> 2 3 4 6
граф НЕ эйлеров, выполняется модификация:
+ ребро (2, 3) уже существует; добавлены рёбра (2, 0) и (0, 3)
+ добавлено ребро (4, 6)
модификация завершена – все степени чётные

матрица смежности после модификации:
      0  1  2  3  4  5  6  7  8  9
0|  0  1  1  1  0  1  1  0  0  1
1|  1  0  1  0  1  0  0  1  0  0
2|  1  1  0  1  1  0  1  1  1  1
3|  1  0  1  0  0  1  0  0  1  0
4|  0  1  1  0  0  0  1  0  0  1
5|  1  0  0  1  0  0  1  1  0  0
6|  1  0  1  0  1  1  0  1  0  1
7|  0  1  1  0  0  1  1  0  1  1
8|  0  0  1  1  0  0  0  1  0  1
9|  1  0  1  0  1  0  1  1  1  0

=== Эйлеров цикл (алгоритм Флёри) ===
длина (рёбер): 26
цикл: 0 -> 9 -> 8 -> 7 -> 9 -> 6 -> 7 -> 5 -> 6 -> 4 -> 9 -> 2 -> 8 -> 3 -> 5 -> 0 -> 6 -> 2 -> 7 -> 1 -> 4 -> 2 -> 3 -> 0 -> 2 -> 1 -> 0

```

Рис. 17: Пункт 14. Пример проверки графа на эйлеровость

Далее на рисунке 18 можно увидеть пример построения фундаментальной системы разрезов.

```

выбор: 15

=== Фундаментальная система разрезов ===
коцикломатическое число  $\pi_k(G) = p - 1 = 9$ 
S1 (удаленное ребро (0-1)): { (0-1) (0-5) (0-6) (0-9) }
S2 (удаленное ребро (1-2)): { (0-5) (1-2) (2-4) (2-6) (2-7) (2-9) (5-6) (5-7) (7-8) (8-9) }
S3 (удаленное ребро (1-4)): { (1-4) (2-4) (4-9) }
S4 (удаленное ребро (1-7)): { (0-6) (0-9) (1-7) (2-6) (2-7) (2-9) (4-9) (5-6) (5-7) (7-8) (8-9) }
S5 (удаленное ребро (2-3)): { (0-5) (2-3) (3-8) (5-6) (5-7) }
S6 (удаленное ребро (2-8)): { (2-8) (3-8) (7-8) (8-9) }
S7 (удаленное ребро (3-5)): { (0-5) (3-5) (5-6) (5-7) }
S8 (удаленное ребро (6-7)): { (0-6) (0-9) (2-6) (2-9) (4-9) (5-6) (6-7) (7-9) (8-9) }
S9 (удаленное ребро (6-9)): { (0-9) (2-9) (4-9) (6-9) (7-9) (8-9) }

получение разреза через симметрическую разность фундаментальных.
сколько фундаментальных разрезов скомбинировать? (0 - пропустить): 3
введите номер разреза (1..9): 1
введите номер разреза (1..9): 2
введите номер разреза (1..9): 3

результат (симметрическая разность выбранных разрезов): { (0-1) (0-6) (0-9) (1-2) (1-4) (2-6) (2-7) (2-9) (4-9) (5-6) (5-7) (7-8) (8-9) }

```

Рис. 18: Пункт 15. Пример построения фундаментальной системы разрезов

При попытке некорректного ввода программа выводит соответствующее сообщение и запрашивает корректный повторный ввод. (см. 19, 20)

```

меню:
===== Lab 1 =====
1. сгенерировать граф
2. сгенерировать весовую матрицу
3. сгенерировать матрицы стоимостей и пропускных способностей
4. найти центр и диаметр
5. метод шимбелла
6. подсчет маршрутов
===== Lab 2 =====
7. обход графа в глубину (dfs)
8. поиск кратчайшего пути (Дейкстра)
===== Lab 3 =====
9. алгоритм Форда-Фалкерсона
10. поток минимальной стоимости [2/3 * max]
===== Lab 4 =====
11. найти число остовных деревьев по т. Кирхгофа
12. минимальный остов (алгоритм Краскала)
13. максимальное независимое множество вершин
===== Lab 5 =====
14. проверка на эйлеров граф
15. фундаментальная система разрезов
0. выход
выбор: 42
неверный выбор

```

Рис. 19: Пример некорректного ввода 1

```

меню:
===== Lab 1 =====
1. сгенерировать граф
2. сгенерировать весовую матрицу
3. сгенерировать матрицы стоимостей и пропускных способностей
4. найти центр и диаметр
5. метод шимбелла
6. подсчет маршрутов
===== Lab 2 =====
7. обход графа в глубину (dfs)
8. поиск кратчайшего пути (Дейкстра)
===== Lab 3 =====
9. алгоритм Форда-Фалкерсона
10. поток минимальной стоимости [2/3 * max]
===== Lab 4 =====
11. найти число остовных деревьев по т. Кирхгофа
12. минимальный остов (алгоритм Краскала)
13. максимальное независимое множество вершин
===== Lab 5 =====
14. проверка на эйлеров граф
15. фундаментальная система разрезов
0. выход
выбор: 9
доступные истоки (нет входящих рёбер): 0 1 2
введите источник: 4
вершина 4 не является истоком

```

Рис. 20: Пример некорректного ввода 2

Заключение

В ходе выполнения данной курсовой работы была разработана программа на языке C++ для работы с графами, реализующая пять лабораторных работ в соответствии с заданием варианта 32.

В первой лабораторной работе реализована генерация связных ациклических графов с использованием распределения Пирсона для генерации степеней вершин, вычислены эксцентриситеты вершин с определением центра и диаметральных вершин, реализован метод Шимбелла для нахождения минимальных и максимальных путей, а также подсчёт количества простых маршрутов между двумя вершинами.

Во второй лабораторной работе реализован алгоритм обхода рёбер графа в глубину с подсчётом количества итераций, алгоритм Дейкстры для нахождения кратчайшего пути между двумя вершинами.

В третьей лабораторной работе сгенерированы матрицы пропускных способностей и стоимостей, реализован алгоритм Форда — Фалкерсона для нахождения максимального потока и алгоритм поиска потока заданной величины минимальной стоимости с использованием алгоритма Дейкстры для поиска кратчайших увеличивающих путей.

В четвёртой лабораторной работе реализована матричная теорема Кирхгофа для подсчёта числа остовных деревьев через вычисление определителя методом Гаусса, алгоритм Краскала для построения минимального остовного дерева, кодирование и декодирование кода Прюфера с сохранением весов рёбер, а также алгоритм нахождения максимального независимого вершинного покрытия.

В пятой лабораторной работе реализована проверка графа на эйлеровость по чётности степеней вершин, модификация графа для приведения к эйлерову путём добавления рёбер между нечётными вершинами, построение эйлерова цикла алгоритмом Флери, построение фундаментальной системы разрезов на основе минимального остова и операция симметрической разности для получения остальных разрезов.

Достоинства программы

1. Реализация пунктов меню разделена на интерфейсную и алгоритмическую часть, что позволяет использовать один и тот же алгоритм для решения более одной задачи, поставленной в работе. Так, например, алгоритмы Дейкстры, Краскала, Форда-Фалкерсона разделены на функции `dijkstra`, `kruskal`, `fordFulkerson` и `dijkstraUI`, `kruskalUI`, `fordFulkersonUI` соответственно.
2. универсальность хранения графа — структура `MyGraph` поддерживает одновременно вес, пропускную способность и стоимость ребра, что позво-

ляет использовать одну структуру для всех лабораторных работ;

3. Использование матриц смежности обеспечивает доступ к элементам за $O(1)$.

Недостатки программы

1. Высокая алгоритмическая сложность алгоритма для матрицы Кирхгофа ($O(n^3)$), ограничивающая применимость на графах с большим числом вершин; Алгоритм Дейкстры, в худшем случае, имеет экспоненциальную сложность $O(2^V)$, когда не заданы ограничения по весам ребер, а исходный граф не содержит отрицательных циклов.
2. Отсутствие возможности для пользователя задавать параметры распределения.

Масштабируемость программы

1. Добавление других алгоритмов, таких как алгоритм Флойда-Уоршала, Беллмана-Форда, Прима, Диница.
2. Реализация одновременного хранения ориентированного и неориентированного графа, что позволит не пересоздавать граф заново при работе алгоритмов для разных типов графа;
3. Реализация пользовательского ввода параметров распределения.

Список литературы

1. Новиков Ф. А. Дискретная математика для программистов. — 3-е изд. — СПб.: Питер, 2009. — 384 с.
2. Секция “Телематика”. URL: <https://tema.spbstu.ru/tgraph/> (дата обращения: 01.05.2026).
3. Базовые алгоритмы на графах // Хабр. URL: <https://habr.com/ru/companies/timeweb/articles/751762/> (дата обращения: 12.06.2026).
4. Эриксон Дж. Алгоритмы. — 2019. — 472 с.